

JUST_BitWarriors Team

Notebook

Jashore University of Science and Technology

Contents

1 Setup & Basics	1
1.1 Sublime Build	1
1.2 Basic Setup	2
1.3 Fast I/O	2
1.4 Debug	2
2 Math	2
2.1 Math Formula Cheatsheet	2
2.2 Algebraic Identities	2
2.3 Inequalities	2
2.4 Probability & Expected Value	2
2.5 Matrix Exponentiation	2
2.6 Basis Vector (XOR Basis)	3
2.7 Linear Combination Checking	3
2.8 FFT (Fast Fourier Transform)	3
2.9 NTT (Number Theoretic Transform)	4
2.10 FWHT	4
2.11 Subset Convolution	4
2.12 Berlekamp-Massey	4
3 Number Theory	5
3.1 GCD Properties	5
3.2 Extended Euclid, CRT, Diophantine	5
3.3 SPF, Möbius, Primes	5
3.4 Segmented Sieve	5
3.5 Miller-Rabin Primality Test	6
3.6 Pollard Rho (Fast Factor)	6
3.7 Phi (Euler Totient)	6
3.8 Möbius Function and Inversion	6
3.9 Gauss Circle Theorem	6
3.10 Sum of Floor(n/i)	6
3.11 Floor Sum	6
3.12 Discrete Logarithm (Baby-Step Giant-Step)	7
3.13 Lifting The Exponent (LTE)	7
3.14 Fibonacci Number Properties	7
4 Combinatorics	7
4.1 Binomial Coefficient Properties	7
4.2 Stars and Bars	7
4.3 Catalan Numbers C_n	7
4.4 Derangements $D(n)$	7
4.5 Partition Numbers $P(n, k)$	7
4.6 Bell Numbers $B(n)$	7

4.7 Stirling 1st Kind $S_1(n, k)$	7
4.8 Stirling 2nd Kind $S_2(n, k)$	8
4.9 Eulerian Numbers $E(n, k)$	8
4.10 Bernoulli & Faulhaber's	8
4.11 Lucas Theorem	8
4.12 Burnside's / Polya	8
4.13 Bishop Placement	8
4.14 Generating Functions	8
5 Data Structure	8
5.1 PBDS	8
5.2 GP Hash Table	8
5.3 DSU	9
5.4 Sparse Table	9
5.5 Sparse Table 2D	9
5.6 Fenwick Tree	9
5.7 Fenwick 2D	9
5.8 BIT Range Update & Range Query	10
5.9 Lazy Segment Tree	10
5.10 Persistent Segment Tree	10
5.11 Merge Sort Tree	11
5.12 Wavelet Matrix	11
5.13 Trie (Binary)	11
5.14 Mo's Algorithm	12
5.15 Rollback DSU	12
5.16 Disjoint Sparse Table	12
5.17 Min Queue	12
5.18 Small-to-Large Merging	13
6 Strings	13
6.1 String Double Hashing	13
6.2 Dynamic String Hash	13
6.3 LPS and SPS	13
6.4 Z-Algorithm	14
6.5 Manacher's Algorithm	14
6.6 Suffix Array	14
6.7 Aho-Corasick	14
6.8 Booth's Algorithm (Min Lex Rotation)	14
6.9 Periods	15
6.10 KMP Automaton	15
6.11 Lyndon Factorization (Duval)	15
6.12 Eertree (Palindromic Tree)	15
6.13 k-th Multiset Permutation	15
7 Graph Theory	15
7.1 Floyd-Warshall	15
7.2 Bellman-Ford / SPFA	15
7.3 Eulerian Path (Directed)	16
7.4 Graph Condense (Kosaraju SCC)	16
7.5 2-SAT	16
7.6 Articulation Points & Bridges	16
7.7 Biconnected Comp. & Bridge Tree	17
7.8 Euler Tour (Tree Flattening)	17
7.9 Binary Lifting	17
7.10 Heavy Light Decomposition (HLD)	17
7.11 Centroid Decomposition	18
7.12 DSU on Tree (Sack)	18
7.13 Hopcroft-Karp	18
7.14 Dinic's Algorithm (Max Flow)	19

7.15 Min Cost Max Flow	19
7.16 Link-Cut Tree	19
8 Geometry	20
8.1 Precision Handling	20
8.2 Geometry Formulas	20
8.3 Geometry Theorems	20
8.4 Closest Pair of Points	20
8.5 Rectangle Distance	20
8.6 Polygon Operations	20
8.7 Area Enclosed by Three Circles	21
8.8 Half-Plane Intersection	21
8.9 Rotating Calipers	22
8.10 Point-in-Convex Polygon ($O(\log N)$)	22
8.11 Minimum Enclosing Circle (Welzl)	22
8.12 Minkowski Sum of Convex Polygons	22
8.13 Circle Operations	23
8.14 Line Sweep (Rectangle Union Area)	23
9 Dynamic Programming	23
9.1 SOS DP (Sum Over Subsets)	23
9.2 Knuth Optimization	23
9.3 Max Rectangular Sum in Matrix	24
9.4 Subset Sum (Bitset)	24
9.5 Exclusion DP (GCD/LCM counting)	24
9.6 Li Chao Tree (CHT)	24
10 Others	24
10.1 Bit Hacks	24
10.2 Zobrist Hashing (XOR Hashing)	24
10.3 Range OR, XOR, AND Tricks	25
10.4 Inclusion-Exclusion with Bitmasks	25
10.5 Sum of Digits up to N	25
10.6 Number of Digits up to N	25
10.7 Game Theory & Nim	25
10.8 Sprague-Grundy	25
1 Setup & Basics	

1.1 Sublime Build

Linux:

```
{
  "shell": true,
  "cmd": "g++ -std=c++17 '${file}' -o
'${file_path}/${file_base_name}' &&
'${file_path}/${file_base_name}' < input.txt >
output.txt",
  "working_dir": "${file_path}",
  "selector": "source.cpp"
}
```

Windows:

```
{
  "cmd": ["g++.exe",
"-std=c++17", "${file}",
"-o",
"${file_base_name}.exe",
"&&" ,
"${file_base_name}.exe<input.txt>output.txt"],
  "shell": true,
  "working_dir": "${file_path}",
}
```

```
"selector":"source.cpp"
```

1.2 Basic Setup

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
#pragma GCC optimize("Ofast")
ios_base::sync_with_stdio(false), cin.tie(NULL),
cout.tie(NULL);
#ifdef ONLINE_JUDGE
freopen("input.txt", "r",
stdin),freopen("output.txt", "w", stdout);
#endif
auto rd = [](){ ll x; cin >> x; return x; };
#define dbg(x) cerr << "[" #x "]" = " << (x) << "\n"
mt19937_64 rnd(chrono::steady_clock::now()
               .time_since_epoch().count());
int64_t Random(int64_t l, int64_t r) {
    return uniform_int_distribution<int64_t>(l, r)(rnd);
}
// Usage: Random(l, r) returns uniform int in [l, r]
// Use gp_hash_table<ll, ll, custom_hash> mp for
// O(1) lookups without hash collisions
```

1.3 Fast I/O

Reads ints only (no strings/floats). Use for large input (10^6+). Redirect using `./a.out < input.txt`. **Note:** If you use `readInt()`, do NOT use `cin` or `cout` anywhere else in the code.

```
inline char gc() {
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++];
}
int readInt() {
    int a, c;
    while ((a = gc()) < 40) if (a == 0) return 0;
    bool minus = (a == '-');
    if (minus) a = gc();
    unsigned res = a - 48;
    while ((c = gc()) >= 48) res = res * 10 + (c - 48);
    return minus ? -(long long)res : (int)res;
}
```

// ~5x faster than cin/scanf

1.4 Debug

`dbg(x1,x2,...)` prints `x1=value1 || x2=value2 || ...` to `stderr`. Works with integers, strings, pairs, containers (vector, set, map). Disabled with `-DONLINE_JUDGE`.

```
// In main.cpp:
#ifdef ONLINE_JUDGE
#include "trace.cpp"
#else
#define dbg(...)
#endif
```

```
// ---- paste below into trace.cpp ----
```

```
#ifndef ONLINE_JUDGE
#define dbg(...) cerr<<__LINE__<<": ", \
    __dbg(#__VA_ARGS__,__VA_ARGS__), cerr<<"\n"

// primitive types (int,ll,char,string,etc.)
template<class T> void __p(T x) { cerr<<x; }
// pair<T1,T2>
template<class T,class U> void __p(pair<T,U> x)
{ cerr<< "("<<x.first<<","<<x.second<<")"; }
// vector<T>
template<class T> void __p(vector<T> x)
{ cerr<< "{";int _=0;for(auto
    & y:x){if(_++)cerr<<",";__p(y);}cerr<<"}"; }
// set<T>
template<class T> void __p(set<T> x)
{ cerr<< "{";int _=0;for(auto
    & y:x){if(_++)cerr<<",";__p(y);}cerr<<"}"; }
// map<K,V>
template<class T,class U> void __p(map<T,U> x)
{ cerr<< "{";int _=0;for(auto[k,v]:x){if(_++)cerr<<",";
    & __p(k);cerr<<":";__p(v);}cerr<<"}";
    & }
// stack<T>
template<class T> void __p(stack<T> s)
{ vector<T>
    & v;while(v.size()||s.size())v.push_back(s.top()),
    & s.pop();
    reverse(v.begin(),v.end());__p(v); }
// queue<T>
template<class T> void __p(queue<T> q)
{ vector<T> v;while(q.size())v.push_back(q.front()),
    & q.pop();__p(v);
    & }
// priority_queue<T>
template<class T> void __p(priority_queue<T> q)
{ vector<T> v;while(q.size())v.push_back(q.top()),
    & q.pop();__p(v);
    & }
```

```
void __dbg(const char*){}
template<class T,class...U>
void __dbg(const char*s,T x,U...t){
    int i=0;while(s[i]&&s[i]!='(',')')i++;
    cerr.write(s,i)<<"=";__p(x);
    if constexpr(sizeof...(t)) cerr<<" || "
    & __dbg(s+i+1,t...);
}
```

```
#else
```

```
#define dbg(...)
```

```
#endif
```

2 Math

2.1 Math Formula Cheatsheet

Geometric Series:

$$\bullet \sum_{i=a}^b c^i = \frac{c^{b+1}-c^a}{c-1} \quad (c \neq 1)$$

$$\bullet \sum_{i=0}^n c^i = \frac{c^{n+1}-1}{c-1} \quad (c \neq 1)$$

$$\bullet \sum_{i=0}^{\infty} c^i = \frac{1}{1-c} \quad (|c| < 1)$$

Sums of Powers:

$$\bullet \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\bullet \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\bullet \sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

$$\bullet \sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

Harmonic Sums:

$$\bullet H_n = \sum_{i=1}^n \frac{1}{i}$$

$$\bullet \sum_{i=1}^n i H_i = \frac{n(n+1)}{2} H_n - \frac{n(n-1)}{4}$$

Combinatorial Sum:

$$\bullet \sum_{k=0}^n \binom{r+k}{k} = \binom{r+n+1}{n}$$

2.2 Algebraic Identities

$$\text{Squares: } (a \pm b)^2 = a^2 \pm 2ab + b^2 \quad | \quad a^2 - b^2 = (a+b)(a-b)$$

$$\text{Cubes: } (a \pm b)^3 = a^3 \pm 3a^2b + 3ab^2 \pm b^3 \quad | \quad a^3 \pm b^3 = (a \pm b)(a^2 \mp ab + b^2)$$

$$\text{Trinomial: } (a+b+c)^2 = a^2 + b^2 + c^2 + 2(ab+bc+ca)$$

$$\text{4th Power: } a^4 + b^4 = (a^2 + b^2)^2 - 2a^2b^2$$

$$\text{n-th Power: } a^n - b^n = (a-b)(a^{n-1} + a^{n-2}b + \dots + b^{n-1})$$

2.3 Inequalities

$$\text{AM-GM: } \frac{x_1 + \dots + x_n}{n} \geq \sqrt[n]{x_1 \dots x_n}$$

$$\text{Cauchy-Schwarz: } (\sum a_i^2)(\sum b_i^2) \geq (\sum a_i b_i)^2$$

$$\text{Triangle: } ||a| - |b|| \leq |a \pm b| \leq |a| + |b|$$

$$\text{Jensen: } f\left(\frac{\sum x_i}{n}\right) \leq \frac{\sum f(x_i)}{n} \text{ for convex } f$$

$$\text{Bernoulli: } (1+x)^n \geq 1+nx \text{ for } n \geq 0, x > -1$$

$$\text{Chebyshev: } \frac{1}{n} \sum a_i b_i \geq \left(\frac{1}{n} \sum a_i\right) \left(\frac{1}{n} \sum b_i\right) \text{ when sorted similarly}$$

$$\text{Hölder: } \sum |x_i y_i| \leq (\sum |x_i|^p)^{1/p} (\sum |y_i|^q)^{1/q} \text{ where } 1/p + 1/q = 1$$

2.4 Probability & Expected Value

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (\text{Bayes})$$

$$E[X] = \sum x \cdot P(X = x)$$

$$E[X + Y] = E[X] + E[Y] \quad (\text{Linearity})$$

$$E[X \cdot Y] = E[X] \cdot E[Y] \text{ if independent}$$

$$Var(X) = E[X^2] - (E[X])^2$$

$$Var(aX + b) = a^2 Var(X)$$

$$\text{Markov: } P(X \geq a) \leq \frac{E[X]}{a}$$

$$\text{Chebyshev: } P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$$

$$\text{Linearity of Expectation: } E[\sum X_i] = \sum E[X_i] \text{ always}$$

$$Cov(X, Y) = E[XY] - E[X]E[Y]$$

2.5 Matrix Exponentiation

`multiply(a,b)` multiplies matrices $O(NMP)$. `expo(t,p)` computes t^p in $O(N^3 \log p)$. Skips zero entries in first matrix for speed. Define `mod` globally. Use for linear recurrences.

```
vector<vector<int>>> multiply(
    vector<vector<int>>& a,
    vector<vector<int>>& b) {
    int n = sz(a), m = sz(a[0]), p = sz(b[0]);
    vector<vector<int>> result(n,
        vector<int>(p, 0));
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < m; k++) {
            if (a[i][k] == 0) continue;
            for (int j = 0; j < p; j++)
                result[i][j] = ((long long)result[i][j] +
                    (long long)a[i][k] * b[k][j]) % mod;
        }
    }
    return result;
}

vector<vector<int>>> expo(
    vector<vector<int>> t, int p) {
    int n = t.size();
    vector<vector<int>> result(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) result[i][i] = 1;
    assert(p >= 0);
    while (p > 0) {
        if (p & 1) result = multiply(result, t);
        t = multiply(t, t);
        p >>= 1;
    }
    return result;
}
```

2.6 Basis Vector (XOR Basis)

Maintains linear basis of vectors mod 2 (XOR space).

insert(x) – returns true if x is independent.

maxXor() – max XOR subset of inserted values.

kth(k) – k-th smallest XOR (1-indexed).

count_less(x) – count of subsets with XOR < x.

count_leq(x) – count of subsets with XOR ≤ x.

can(x) – checks if x is representable.

reset() – clears the basis for reuse.

jumpK(val, k) – k-th XOR value ≥ val (rank + k).

template<typename T = int, int LG = 31>

```
struct Basis {
    T basis[LG]; int sz;
    vector<int> who;
    Basis() { reset(); }
    void reset() {
        fill(basis, basis + LG, 0);
        sz = 0; who.clear();
    }
    bool insert(T mask, int idx = -1) {
        for (int i = LG - 1; i >= 0; --i) {
            if (mask >> i & 1) {
                if (basis[i]) mask ^= basis[i];
                else { basis[i] = mask; ++sz;
                    who.push_back(idx); return true; }
            }
        }
        return false;
    }
}
```

```
}
T maxXor() {
    T ans = 0;
    for (int i = LG - 1; i >= 0; --i)
        ans = max(ans, ans ^ basis[i]);
    return ans;
}
bool can(T x) {
    for (int i = LG - 1; i >= 0; --i) {
        if (x >> i & 1) {
            if (!basis[i]) return false;
            x ^= basis[i];
        }
    }
    return true;
}
T kth(T k) {
    if (k < 1 || (uint64_t)k > (1ULL << sz)) return -1;
    T mask = 0;
    int64_t tot = 1LL << sz;
    for (int i = LG - 1; i >= 0; --i) {
        if (basis[i]) {
            int64_t low = tot / 2;
            if ((low < k && (~mask >> i & 1)) ||
                (low >= k && (mask >> i & 1)))
                mask ^= basis[i];
            if (low < k) k -= low;
            tot /= 2;
        }
    }
    return mask;
}
T count_less(T x) {
    if (x < 0) return 0;
    T ret = 0, mask = 0;
    uint64_t cnt = 1ULL << sz;
    for (int i = LG - 1; i >= 0; i--) {
        if (basis[i]) {
            if (x >> i & 1) {
                ret += (cnt >> 1);
                if (~mask >> i & 1) mask ^= basis[i];
            } else {
                if (mask >> i & 1) mask ^= basis[i];
            }
            cnt >>= 1;
        } else {
            if ((x ^ mask) >> i & 1) {
                if (x >> i & 1) return ret + cnt;
                else return ret;
            }
        }
    }
    return ret;
}
T count_leq(T x) { return count_less(x) + (can(x) ? 1 : 0); }
T jumpK(T val, T k) {
    return kth(count_leq(val) + k);
}
```

```
}
```

```
};
```

2.7 Linear Combination Checking

Given array `arr` and integer `N`, check if `N` is a linear combination of `arr`'s elements: $N = a \cdot x + b \cdot y + c \cdot z + \dots$

Ans: $N \bmod \text{GCD}(\text{all elements}) == 0$. **Note:** A sum S can be formed if $S \bmod \text{gcd} == 0$.

2.8 FFT (Fast Fourier Transform)

Convolution for integer polynomials (double-based). `multiply(a,b)` returns $a * b$ as `vector<long long>`. Prefer NTT when modulus supports it (no precision issues). Complexity $O(N \log N)$ where N is next power of 2. **Note:** Array sizes must be a power of 2. Pad arrays with 0s before calling.

```
const double PI = acos(-1);
struct base {
    double a, b;
    base(double a = 0, double b = 0) : a(a), b(b) {}
    const base operator+(const base &c) const {
        return base(a + c.a, b + c.b);
    }
    const base operator-(const base &c) const {
        return base(a - c.a, b - c.b);
    }
    const base operator*(const base &c) const {
        return base(a*c.a - b*c.b, a*c.b + b*c.a);
    }
};

void fft(vector<base> &p, bool inv = 0) {
    int n = p.size(), i = 0;
    for (int j = 1; j < n - 1; ++j) {
        for (int k = n >> 1; k > (i ^= k); k >>= 1);
        if (j < i) swap(p[i], p[j]);
    }
    for (int l = 1; (l << 1) <= n; l <<= 1) {
        double ang = 2 * PI / (l << 1);
        base wn(cos(ang), (inv ? 1 : -1) * sin(ang)), w;
        for (int i = 0; i < n; i += l << 1) {
            w = base(1, 0);
            for (int j = i, k = i + l; j < k; ++j, w = w * wn) {
                base t = w * p[j + l];
                p[j + l] = p[j] - t;
                p[j] = p[j] + t;
            }
        }
    }
    if (inv)
        for (int i = 0; i < n; ++i)
            p[i].a /= n, p[i].b /= n;
}

vector<long long> multiply(vector<long long> &a,
    vector<long long> &b) {
    if (a.empty() || b.empty()) return {};
    int n = a.size(), m = b.size(), t = n + m - 1, sz =
        1;
    while (sz < t) sz <<= 1;
}
```

```
vector<base> x(sz), y(sz);
for (int i = 0; i < sz; ++i) {
    x[i] = i < n ? base[a[i], 0] : base(0, 0);
    y[i] = i < m ? base[b[i], 0] : base(0, 0);
}
fft(x), fft(y);
for (int i = 0; i < sz; ++i) x[i] = x[i] * y[i];
fft(x, 1);
vector<long long> ret(sz);
for (int i = 0; i < sz; ++i)
    ret[i] = (long long)round(x[i].a);
while (ret.size() > 1 && ret.back() == 0)
    ret.pop_back();
return ret;
}
```

2.9 NTT (Number Theoretic Transform)

Modular convolution (mod998244353, primitive root = 62). Inputs must be in $[0, \text{mod})$ before calling. `conv(a,b)` returns polynomial product modulo MOD. Replace MOD and ROOT for different moduli (mod = $c \cdot 2^k + 1$). **Note:** The modulo must be of the form $c \cdot 2^k + 1$ (like 998244353).

```
const int MOD = 998244353, ROOT = 62;
int modpow(int a, int e) {
    int r = 1;
    while (e) {
        if (e & 1) r = (long long)r * a % MOD;
        a = (long long)a * a % MOD;
        e >>= 1;
    }
    return r;
}
void ntt(vector<int> &a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<int> rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        int z[] = {1, modpow(ROOT, MOD >> s)};
        for (int i = k; i < 2 * k; i++)
            rt[i] = (long long)rt[i / 2] * z[i & 1] % MOD;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                int z = (long long)rt[j + k] * a[i + j + k] %
                    MOD;
                a[i + j + k] = a[i + j] - z +
                    (z > a[i + j] ? MOD : 0);
                a[i + j] += a[i + j] + z >= MOD ? z - MOD : z;
            }
}
vector<int> conv(const vector<int> &a,
```

```
const vector<int> &b) {
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1,
        n = 1 << (32 - __builtin_clz(s));
    int inv = modpow(n, MOD - 2);
    vector<int> L(a), R(b), out(n);
    L.resize(n); R.resize(n);
    ntt(L); ntt(R);
    for (int i = 0; i < n; i++)
        out[(n - i) & (n - 1)] = (long long)L[i] * R[i] %
            MOD * inv % MOD;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

2.10 FWHT

Convolution where $c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$. type: 0=OR, 1=AND, 2=XOR. inv=true for inverse. modpow(n, MOD-2) for inverse normalization (type 2 only). **Note:** Array size must be a power of 2.

```
void fwht(vector<int> &a, bool inv, int type) {
    int n = a.size();
    for (int len = 1; len < n; len <= 1)
        for (int i = 0; i < n; i += 2 * len)
            for (int j = 0; j < len; j++) {
                int u = a[i + j], v = a[i + len + j];
                if (type == 2) {
                    a[i + j] = (u + v) % MOD;
                    a[i + len + j] = (u - v + MOD) % MOD;
                } else if (type == 1) {
                    a[i + j] = inv ? (u - v + MOD) % MOD : (u + v) % MOD;
                } else {
                    a[i + len + j] = inv ? (v - u + MOD) % MOD : (u + v) % MOD;
                }
            }
    if (inv && type == 2) {
        int inv_n = modpow(n, MOD - 2);
        for (auto &x : a) x = (ll)x * inv_n % MOD;
    }
}
// conv(a, b) = for type=0: c[k]=sum_{i|j=k} a[i]*b[j]
// for type=1: c[k]=sum_{i&j=k} a[i]*b[j]
// for type=2: c[k]=sum_{i^j=k} a[i]*b[j]
2.11 Subset Convolution
Computes  $h[mask] = \sum_{sub \subseteq mask} f[sub] \cdot g[mask \setminus sub]$  where sub and mask \setminus sub are disjoint. Unlike FWHT OR-convolution (which allows overlap), subset convolution enforces disjointness via an extra dimension for population count.  $O(B^2 \cdot 2^B)$ . When to use: Set partition DP, graph matching on small graphs, counting k-colorings, any DP where two disjoint subsets are combined.
vector<int> subset_conv(const vector<int> &f,
const vector<int> &g, int B) {
    int M = 1 << B;
    vector<vector<int>> F(B+1, vector<int>(M));
    vector<vector<int>> G(B+1, vector<int>(M));
```

```
vector<vector<int>> H(B+1, vector<int>(M));
for (int m = 0; m < M; m++) {
    int p = __builtin_popcount(m);
    F[p][m] = f[m]; G[p][m] = g[m];
}
// Zeta transform (SOS) on each rank
for (int k = 0; k <= B; k++)
    for (int b = 0; b < B; b++)
        for (int m = 0; m < M; m++)
            if (m >> b & 1) {
                F[k][m] = (F[k][m] + F[k][m ^ (1<<b)]) % MOD;
                G[k][m] = (G[k][m] + G[k][m ^ (1<<b)]) % MOD;
            }
// Pointwise multiply over ranks
for (int m = 0; m < M; m++)
    for (int k = 0; k <= B; k++)
        for (int i = 0; i <= k; i++)
            H[k][m] = (H[k][m] +
                (ll)F[i][m] * G[k-i][m]) % MOD;
// Mobius transform (inverse zeta)
for (int k = 0; k <= B; k++)
    for (int b = 0; b < B; b++)
        for (int m = 0; m < M; m++)
            if (m >> b & 1)
                H[k][m] = (H[k][m] - H[k][m ^ (1<<b)]
                    + MOD) % MOD;
vector<int> h(M);
for (int m = 0; m < M; m++)
    h[m] = H[__builtin_popcount(m)][m];
return h;
}
```

2.12 Berlekamp-Massey

What it is: A "black box" algorithm that looks at a sequence of numbers (like the first few terms of a DP) and automatically discovers the hidden formula (linear recurrence) generating them. E.g., for Fibonacci, it discovers $a_n = a_{n-1} + a_{n-2}$.

How to use: Compute the first ≈ 100 terms of your sequence using naive DP and pass them as a vector. It returns the recurrence coefficients *C*. You can then use Matrix Exponentiation (or Kitamasa) with *C* to find the *K*-th term for a massive *K*.

Note: Needs at least $2L$ terms to correctly find a recurrence of length *L*.

```
vector<int> berlekampMassey(vector<int> x) {
    int n = x.size(), L = 0, m = 1;
    ll b = 1;
    vector<int> C(n), B(n);
    C[0] = B[0] = 1;
    for (int i = 0; i < n; i++, m++) {
        ll d = x[i] % MOD;
        for (int j = 1; j <= L; j++)
            d = (d + (ll)C[j] * x[i - j]) % MOD;
        if (!d) continue;
        vector<int> T = C;
```

```

11 c = d * modpow(b, MOD - 2) % MOD;
for (int j = m; j < n; j++)
    C[j] = (C[j] - c * B[j - m] + MOD) % MOD;
if (2 * L <= i) {
    L = i - L + 1; B = T;
    b = d; m = 0;
}
}
C.resize(L + 1); C.erase(C.begin());
for (auto &i : C) i = (MOD - i) % MOD;
return C;
}
// Kitamasa: compute k-th term of linear recurrence
// using C (from BM) and first terms.
// Complexity O(L^2 log k).

```

3 Number Theory

3.1 GCD Properties

- **Divisibility Trick:** If a perfectly divides $(b \times c)$, and the GCD of a and b is d , then the remaining part of a (which is a/d) MUST perfectly divide c .
- **Exponents:** The GCD of $(n^a - 1)$ and $(n^b - 1)$ is beautifully simple: it is just $n^{\gcd(a,b)} - 1$.
- **Geometry:** If you draw a straight line on a 2D grid from $(0,0)$ to (a,b) , the number of exact grid points strictly between them is $\gcd(|a|, |b|) - 1$.

```

// Inline x86 GCD via idiv / faster than __gcd on old
// compilers.
// Avoid for large inputs; prefer std::gcd (C++17)
// otherwise.

```

```

int gcdFast(int a, int b) {
    int result;
    __asm__ __volatile__(
        "movl %1, %%eax;"
        "movl %2, %%ebx;"
        "CONTD: cmpl $0, %%ebx;"
        "je DONE;"
        "xorl %%edx, %%edx;"
        "idivl %%ebx;"
        "movl %%ebx, %%eax;"
        "movl %%edx, %%ebx;"
        "jmp CONTD;"
        "DONE: movl %%eax, %0;"
        : "=g"(result)
        : "g"(a), "g"(b));
    return result;
}

```

3.2 Extended Euclid, CRT, Diophantine

$\text{egcd}(a,b,x,y)$ returns $g = \gcd(a,b)$ with $ax + by = g$. $\text{crt}(a_1,m_1,a_2,m_2)$ solves $x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}$. Returns $\{x, \text{lcm}(m_1, m_2)\}$ or $\{0, -1\}$ if no solution. $\text{find_any_solution}(a,b,c,x_0,y_0,g)$ solves $Ax + By = C$. Solutions exist iff $g = \gcd(|A|, |B|)$ divides C . General:

$x = x_0 + k \frac{B}{g}$, $y = y_0 - k \frac{A}{g}$. **Note:** CRT requires pairwise coprime moduli. If not coprime, it returns $\{0, -1\}$.

```

using T = __int128_t;
T egcd(T a, T b, T &x, T &y) {
    T xx = y = 0, yy = x = 1;
    while (b) {
        T q = a / b;
        T t = b; b = a % b; a = t;
        t = xx; xx = x - q * xx; x = t;
        t = yy; yy = y - q * yy; y = t;
    }
    if (a < 0) { a = -a; x = -x; y = -y; }
    return a;
}

```

```

pair<T, T> crt(T a1, T m1, T a2, T m2) {
    T p, q;
    T g = egcd(m1, m2, p, q);
    if (a1 % g != a2 % g) return {0, -1};
    T m = m1 / g * m2;
    p = (p % m + m) % m;
    q = (q % m + m) % m;
    T x = p * a2 % m * (m1 / g) % m +
        q * a1 % m * (m2 / g) % m;
    return {x % m, m};
}

```

```

11 extgcd(11 a, 11 b, 11 &x, 11 &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    11 x1, y1;
    11 d = extgcd(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b);
    return d;
}

```

```

bool find_any_solution(11 a, 11 b, 11 c,
11 &x0, 11 &y0, 11 &g) {
    g = extgcd(abs(a), abs(b), x0, y0);
    if (c % g) return false;
    x0 *= c / g; y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

```

3.3 SPF, Möbius, Primes

```

// O(N)
const int N = 1e6 + 9;
int spf[N], mob[N];
vector<int> primes;
void sieve() {
    mob[1] = 1;
    for (int i = 2; i < N; i++) {
        if (!spf[i]) spf[i] = i, mob[i] = -1,
            primes.push_back(i);
        for (int p : primes) {
            if (p > spf[i] || 1LL * i * p >= N) break;
            spf[i * p] = p;
            if (i % p == 0) { mob[i * p] = 0; break; }
            mob[i * p] = -mob[i];
        }
    }
}

```

```

}
}
// factorize x (x <= (N-1)^2)
vector<pair<int,int>> factorize(int x) {
    vector<pair<int,int>> ret;
    for (int p : primes) {
        if (1LL * p * p > x) break;
        if (x % p == 0) { int c = 0; while (x % p == 0) x
            /= p, c++; ret.emplace_back(p, c); }
    }
    if (x > 1) {
        if (x >= N) ret.emplace_back(x, 1);
        else { while (x > 1) { int p = spf[x], c = 0; while
            (x % p == 0) x /= p, c++; ret.emplace_back(p,
            c); } }
    }
    return ret;
}
// divisors of x (x <= (N-1)^2)
vector<int> divisors(int x) {
    vector<int> ret = {1};
    for (auto [p, c] : factorize(x)) {
        int n = ret.size(); long long a = 1;
        for (int d = 1; d <= c; d++) { a *= p; for (int i
            = 0; i < n; i++) ret.push_back(ret[i] * a); }
    }
    return ret;
}

```

3.4 Segmented Sieve

Key idea: Sieve small primes $\leq \sqrt{R}$, then strike composites in $[L, R]$ using offsets. Efficient for very large ranges ($L, R \leq 10^{12}$, $R - L \leq 10^7$).

```

vector<char> segmentedSieve(long long L,
long long R) {
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }
    vector<char> isPrime(R - L + 1, true);
    for (long long i : primes) {
        11 sm = (L / i) * i;
        if (sm < L) sm += i;
        for (long long j = max(i * i, sm); j <= R; j += i)
            isPrime[j - L] = false;
    }
    if (L <= 1) {
        for (long long i = L; i <= 1 && i <= R; i++)
            isPrime[i - L] = false;
    }
    return isPrime;
}

```

3.5 Miller-Rabin Primality Test

Deterministic for all 64-bit integers. $O(\log^3 n)$. Uses fixed bases $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$. Returns `true` if n is prime, `false` if composite.

```
ull pow(ull a, ull t, ull mod) {
    ull r = 1;
    for (; t >= 1, a = (__uint128_t)a * a % mod)
        if (t & 1) r = (__uint128_t)r * a % mod;
    return r;
}
bool isprime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775,
        9780504, 1795265022};
    s = __builtin_ctzll(n - 1), d = n >> s;
    for (ull a : A) {
        ull p = pow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = (__uint128_t)p * p % n;
        if (p != n - 1 && i != s) return 0;
    }
    return 1;
}
```

3.6 Pollard Rho (Fast Factor)

Factor any 64-bit integer in $O(n^{1/4})$ expected time. Uses Montgomery multiplication for fast modular arithmetic. `find_factor(n)` returns a non-trivial factor (or n if prime). Pair with Miller-Rabin: if `isprime(n)` $\rightarrow$ done, else factor. **Note:** Randomized. Can get stuck in an infinite loop if N is prime, always check `isprime(N)` first.

```
#define ull uint64_t
struct Montgomery {
    ull n, nr;
    Montgomery(ull n) {
        this->n = n, nr = 1;
        for (int i = 0; i < 6; i++)
            nr *= 2 - n * nr;
    }
    ull reduce(__uint128_t x) const {
        ull q = ull(x) * nr,
            m = ((__uint128_t)q * n) >> 64;
        ull res = (x >> 64) + n - m;
        if (res >= n) res -= n;
        return res;
    }
    ull multiply(ull x, ull y) {
        return reduce((__uint128_t)x * y);
    }
};
ull f(ull x, ull n, ull a) {
    return ((__uint128_t)x * x + a) % n;
}
ull diff(ull a, ull b) {
    return a > b ? a - b : b - a;
}
const int M = 1024;
const ull SEED = 42;
ull find_factor(ull n, ull x0 = 2, ull a = 1) {
```

```
    ull x = x0, y = x0, d = 1;
    while (d == 1) {
        x = f(x, n, a);
        y = f(f(y, n, a), n, a);
        d = gcd(diff(x, y), n);
    }
    return d == n ? find_factor(n, x0, a + 1) : d;
}
```

3.7 Phi (Euler Totient)

$\varphi(n)$ = count of integers in $[1, n]$ coprime to n . Formula:
 $\varphi(n) = n \prod_{p|n} (1 - 1/p)$. Computed via trial division.
`phi_1_to_n()` computes φ for all $i \in [1, N]$ using linear sieve:
 $O(N)$.

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            result -= result / i;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}
void phi_1_to_n() {
    phi[1] = 1;
    for (int i = 2; i < N; i++) {
        if (phi[i] == 0) {
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (int p : primes) {
            if (i * p >= N) break;
            if (i % p == 0) {
                phi[i * p] = phi[i] * p; break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
}
```

Sum of numbers $\leq x$ coprime to x : $\sum_{\substack{1 \leq i \leq x \\ \gcd(i, x) = 1}} i = \frac{x \cdot \phi(x)}{2}$.

Euler's Totient Theorem: If $\gcd(a, n) = 1$, $a^{\phi(n)} \equiv 1 \pmod{n}$, and consequently $a^n \equiv a^{n - \phi(n)} \pmod{n}$. For general a, n (not necessarily coprime), for $m \geq \log_2 n$: $a^m \equiv a^{(m \bmod \phi(n)) + \phi(n)} \pmod{n}$.

3.8 Möbius Function and Inversion

The Möbius function $\mu(n)$:

- $\mu(n) = 1$ if n is square-free with even number of prime factors.
- $\mu(n) = -1$ if n is square-free with odd number of prime factors.

- $\mu(n) = 0$ if n has a squared prime factor.

Properties & Inversion:

- $\sum_{d|n} \mu(d) = [n = 1]$
- $g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g(n/d)$
- $\phi(n) = \sum_{d|n} \mu(d) \frac{n}{d}$ and $n = \sum_{d|n} d \cdot \mu\left(\frac{n}{d}\right)$
- $\frac{n}{\phi(n)} = \sum_{d|n} \frac{\mu(d)}{d}$

GCD Sums:

- $\sum_{i=1}^n [\gcd(i, n) = k] = \phi\left(\frac{n}{k}\right)$
- $\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} d \cdot \phi\left(\frac{n}{d}\right)$
- $\sum_{i=1}^n \frac{1}{\gcd(i, n)} = \frac{1}{n} \sum_{d|n} d \cdot \phi(d)$
- $\sum_{i=1}^n \frac{i}{\gcd(i, n)} = \frac{1}{2} \sum_{d|n} d \cdot \phi(d)$

3.9 Gauss Circle Theorem

Lattice points inside $m^2 + n^2 \leq r^2$:

$$N(r) = 1 + 4 \sum_{i=0}^{\infty} \left(\left\lfloor \frac{r^2}{4i+1} \right\rfloor - \left\lfloor \frac{r^2}{4i+3} \right\rfloor \right)$$

Note: Counts integer points (x, y) such that $x^2 + y^2 \leq r^2$.

3.10 Sum of Floor(n/i)

$\sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$ in $O(\sqrt{n})$ by grouping equal terms. **Applications:** Divisor sum $\sum d(i)$, Dirichlet hyperbola basis, $\sum_{i=1}^n \lfloor n/i \rfloor \phi(i)$.

```
ll sum_floor(ll n) {
    ll ans = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans += (r - l + 1) * (n / l);
    }
    return ans;
}
```

3.11 Floor Sum

Computes $\sum_{i=0}^{n-1} \left\lfloor \frac{a \cdot i + b}{m} \right\rfloor$ in $O(\log m)$ via Euclidean recursion. From AtCoder Library (ACL). **Applications:** Lattice counting, $\sum (a \cdot i + b) \% m$, digit DP.

```
ll floor_sum(ll n, ll m, ll a, ll b) {
    ll ans = 0;
    if (a >= m) {
        ans += (n - 1) * n * (a / m) / 2;
        a %= m;
    }
    if (b >= m) {
        ans += n * (b / m);
        b %= m;
    }
    ll y_max = (a * n + b) / m;
    if (y_max == 0) return ans;
    ans += floor_sum(y_max, a, m, (a * n + b) % m);
    return ans;
}
```

3.12 Discrete Logarithm (Baby-Step Giant-Step)

Solves $a^x \equiv b \pmod{m}$ for smallest $x \geq 0$. Requires $\gcd(a, m) = 1$. $O(\sqrt{m})$ meet-in-the-middle.

```
ll discrete_log(ll a, ll b, ll m) {
    a %= m; b %= m;
    if (b == 1) return 0;
    ll n = sqrt(m) + 1;
    unordered_map<ll, ll> baby;
    ll cur = b;
    for (ll q = 0; q < n; q++) {
        baby[cur] = q; // overwrite: keep the largest q
        // for smallest x
        cur = cur * a % m;
    }
    ll an = 1;
    for (ll i = 0; i < n; i++) an = an * a % m;
    cur = 1;
    for (ll p = 0; p <= n; p++) {
        if (baby.count(cur)) {
            ll x = p * n - baby[cur];
            if (x >= 0) return x;
        }
        cur = cur * an % m;
    }
    return -1;
}
```

3.13 Lifting The Exponent (LTE)

What it is: Finds the highest power of a prime p dividing $a^n \pm b^n$. $\nu_p(X)$ is the exponent of p in X 's prime factorization. **When to use:** Checking if $p^k \mid a^n \pm b^n$ for huge exponents, or finding trailing zeros. **How to use:** Check strict preconditions ($p \mid a \pm b$, $p \nmid a$, $p \nmid b$), then add powers:

Odd Prime ($p \geq 3$):

- $\nu_p(a^n - b^n) = \nu_p(a - b) + \nu_p(n)$
- $\nu_p(a^n + b^n) = \nu_p(a + b) + \nu_p(n)$ (requires n to be **odd**)

Even Prime ($p = 2$), requires a, b odd:

- $\nu_2(a^n - b^n) = \nu_2(a - b) + \nu_2(a + b) + \nu_2(n) - 1$ (if n is **even**)
- $\nu_2(a^n - b^n) = \nu_2(a - b)$ (if n is **odd**)
- $\nu_2(a^n + b^n) = 1$ (if n is **even**)
- $\nu_2(a^n + b^n) = \nu_2(a + b)$ (if n is **odd**)

3.14 Fibonacci Number Properties

$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$.

- To check if one Fibonacci divides another, just check their indices: $F_m \mid F_n \iff m \mid n$.
- Calculate the GCD of their indices instead: $\gcd(F_m, F_n) = F_{\gcd(m, n)}$.

- Simplifies alternating sum puzzles or determinants: $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$.

- Backbone for $O(\log N)$ doubling/matrix algorithms combining states n, m : $F_{n+m} = F_{n-1}F_m + F_nF_{m+1}$.

- Maps recursive steps that go below zero back to positive: $F_{-n} = (-1)^{n+1}F_n$.

- Eliminates $O(N)$ loops when querying the sum of N terms: $\sum_{i=1}^n F_i = F_{n+2} - 1$.

- Eliminates $O(N)$ loops in grid/tiling sums: $\sum_{i=1}^n F_i^2 = F_nF_{n+1}$.

- For huge N and small modulo m , the sequence repeats. The cycle length is $\leq 6m$. Compute $N \bmod$ cycle.

- $O(1)$ direct calc. Usable under modulo only if 5 is a quadratic residue mod p : $F_n = \frac{\phi^n - \psi^n}{\sqrt{5}}$ ($\phi = \frac{1+\sqrt{5}}{2}$, $\psi = \frac{1-\sqrt{5}}{2}$).

```
// Fast Doubling | returns {F_n, F_{n+1}} in O(log n),
// n up to 1e18
// F_0=0, F_1=1. F_n = fib(n).first
const long long MOD = 1e9 + 7;
pair<long long, long long> fib(long long n) {
    if (n == 0) return {0, 1};
    auto [a, b] = fib(n >> 1);
    long long c = a * ((2 * b % MOD - a + MOD) % MOD) %
    MOD; // F(2k)
    long long d = (a * a % MOD + b * b % MOD) % MOD;
    // F(2k+1)
    if (n & 1) return {d, (c + d) % MOD}; // F(2k+1),
    // F(2k+2)
    return {c, d}; // F(2k),
    // F(2k+1)
}
```

4 Combinatorics

4.1 Binomial Coefficient Properties

Symmetry: $\binom{n}{k} = \binom{n}{n-k}$

Pascal (Addition): $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$

Absorption (Committee): $k\binom{n}{k} = n\binom{n-1}{k-1}$

Sum (Total Subsets): $\sum_{k=0}^n \binom{n}{k} = 2^n$

Alternating Sum: $\sum_{k=0}^n (-1)^k \binom{n}{k} = 0$

Even/Odd Sum: $\sum_{k \text{ even/odd}} \binom{n}{k} = 2^{n-1}$

Hockey-Stick: $\sum_{i=k}^n \binom{i}{k} = \binom{n+1}{k+1}$

Sum over Upper Index: $\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$

Vandermonde's Convolution: $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$

Sum of Squares: $\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$

Weighted: $\sum_{k=1}^n k \binom{n}{k} = n \cdot 2^{n-1}$

Fibonacci: $\sum_{k \geq 0} \binom{n-k}{k} = F_{n+1}$

Multinomial: $\frac{N!}{c_1!c_2!\dots} = \binom{N}{c_1} \binom{N-c_1}{c_2} \dots$

4.2 Stars and Bars

- Ways to put n identical items into k distinct bins (can be empty): $\binom{n+k-1}{k-1}$

- Ways to put n identical items into k distinct bins (non-empty): $\binom{n-1}{k-1}$

4.3 Catalan Numbers C_n

Seq: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862 ...

- Formula: $C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$

- Recurrence: $C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$

- OGF: $\frac{1-\sqrt{1-4x}}{2x}$

- Applications: Correct bracket seqs, polygon triangulations, non-crossing chords, paths $(0, 0) \rightarrow (n, n)$ below diagonal, full binary trees, Dyck words.

4.4 Derangements $D(n)$

Permutations where $\pi(i) \neq i$.

Seq: 1, 0, 1, 2, 9, 44, 265, 1854 ...

- Recurrence: $D(n) = (n-1)(D(n-1) + D(n-2))$

- Formula: $D(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!} \approx \frac{n!}{e}$

- EGF: $\frac{e^{-x}}{1-x}$

- Code:

```
if (n > 0) D[1] = 0;
if (n > 1) D[2] = 1;
for (int i = 3; i <= n; i++)
    D[i] = (i - 1) * (D[i - 1] + D[i - 2]);
```

4.5 Partition Numbers $P(n, k)$

Ways to write n as sum of exactly k positive ints.

- Recurrence: $P(n, k) = P(n-1, k-1) + P(n-k, k)$

- Total partitions: $P(n) = \sum_{k=0}^n P(n, k)$

Seq: 1, 1, 2, 3, 5, 7, 11, 15, 22, 30 ...

4.6 Bell Numbers $B(n)$

Total ways to partition a set of n elements.

Note: Total ways to partition a set of distinct elements.

- $B(n) = \sum_{k=0}^n S_2(n, k)$

- $B(n+1) = \sum_{k=0}^n \binom{n}{k} B(k)$

- EGF: $\exp(\exp(x) - 1)$

4.7 Stirling 1st Kind $S_1(n, k)$

Permutations of n elements with exactly k cycles.

Note: Ways to arrange N elements into exactly K cycles.

- Recurrence: $S_1(n, k) = S_1(n-1, k-1) + (n-1)S_1(n-1, k)$

- Total sum: $\sum_{k=0}^n S_1(n, k) = n!$

- EGF: $(-\ln(1-x))^k / k!$

4.8 Stirling 2nd Kind $S_2(n, k)$

Partition n elements into k non-empty sets.

Note: Ways to partition N distinct elements into K non-empty sets.

- Recurrence: $S_2(n, k) = S_2(n-1, k-1) + k \cdot S_2(n-1, k)$
- Formula: $S_2(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$
- EGF: $(\exp(x) - 1)^k / k!$

4.9 Eulerian Numbers $E(n, k)$

Permutations with exactly k ascents.

Note: Permutations with exactly K ascents (where $p_i < p_{i+1}$).

- Recurrence: $E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$
- Formula: $E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$
- $E(n, k) = E(n, n-1-k)$

4.10 Bernoulli & Faulhaber's

Note: Used to find sum of powers $\sum_{i=1}^N i^k$.

- Recurrence: $B_n = -\frac{1}{n+1} \sum_{k=0}^{n-1} \binom{n+1}{k} B_k$
- EGF: $\frac{x}{e^x - 1}$
- Sum of powers: $\sum_{k=1}^n k^c = \frac{1}{c+1} \sum_{k=0}^c (-1)^k \binom{c+1}{k} B_k n^{c+1-k}$

4.11 Lucas Theorem

Computes $\binom{n}{r} \bmod p$ for prime p , when n, r are large. Breaks n, r into base- p digits and multiplies small binomials. MOD must be prime and $\leq 10^5$ for precomputed factorials. Usage: $C(10^{18}, 10^9) \bmod 1000000007$ in $O(\log_p n)$ time. **Note:** Computes $\binom{N}{R} \bmod P$ for large N, R and small prime P .

```
long long lucas(long long n, long long r) {
    if (r == 0) return 1;
    return lucas(n / MOD, r / MOD) *
           nCr(n % MOD, r % MOD) % MOD;
}
```

4.12 Burnside's / Polya

Counts distinct colorings under group actions. Answer = $\frac{1}{|G|} \sum_{g \in G} (\text{colorings fixed by } g)$. Code: Rotation group of $n \times m$ grid $\times$ reflection. See also: `count_cycles(p)` to compute $k^{c(\sigma)}$. **Note:** Counts distinct objects under symmetry (like rotating a necklace).

```
using Permutation = vector<int>;
void operator*=(Permutation& p, Permutation
const& q) {
    Permutation copy = p;
    for (int i = 0; i < p.size(); i++)
        p[i] = copy[q[i]];
}

int count_cycles(Permutation p) {
    int cnt = 0;
    for (int i = 0; i < p.size(); i++) {
        if (p[i] != -1) {
            cnt++;
            for (int j = i; p[j] != -1;) {
```

```
                int next = p[j]; p[j] = -1; j = next;
            }
        }
    }
    return cnt;
}

int solve(int n, int m) {
    Permutation p(n*m), p1(n*m), p2(n*m), p3(n*m);
    for (int i = 0; i < n*m; i++) {
        p[i] = i;
        p1[i] = (i % n + 1) % n + i / n * n;
        p2[i] = (i / n + 1) % m * n + i % n;
        p3[i] = (m - 1 - i / n) * n + (n - 1 - i % n);
    }
    set<Permutation> s;
    for (int i1 = 0; i1 < n; i1++) {
        for (int i2 = 0; i2 < m; i2++) {
            for (int i3 = 0; i3 < 2; i3++) {
                s.insert(p); p *= p3;
            }
            p *= p2;
        }
        p *= p1;
    }
    int sum = 0;
    for (Permutation const& p : s)
        sum += 1 << count_cycles(p);
    return sum / s.size();
}
```

4.13 Bishop Placement

Count ways to place K bishops on an $N \times N$ chessboard so no two attack each other. Bishops on black and white diagonals are independent, so the two color sets are solved separately via DP and combined. `squares(i)` returns the number of cells on the i -th diagonal. $O(N^2)$. **Note:** Bishops on black squares and white squares do not attack each other.

```
int bishop_placements(int N, int K) {
    if (K > 2 * N - 1) return 0;
    auto len = [&](int i) {
        if (i > N) return 2 * N - i;
        return i;
    };
    // Collect diagonal lengths for each color, sorted
    //   ↪ increasingly
    vector<int> black, white;
    for (int i = 1; i <= 2 * N - 1; i++) {
        if (i & 1) black.push_back(len(i));
        else white.push_back(len(i));
    }
    sort(black.begin(), black.end());
    sort(white.begin(), white.end());
    auto solve = [&](vector<int> &diag, int K) {
        int m = diag.size();
        vector<vector<int>> D(m + 1, vector<int>(K + 1,
        //   ↪ 0));
        D[0][0] = 1;
```

```
        for (int i = 1; i <= m; i++)
            for (int j = 0; j <= K; j++) {
                D[i][j] = D[i-1][j];
                if (j > 0)
                    D[i][j] += D[i-1][j-1] * (diag[i-1] - j + 1);
            }
        return D[m];
    };
    auto B = solve(black, K), W = solve(white, K);
    int ans = 0;
    for (int i = 0; i <= K; i++)
        ans += B[i] * W[K - i];
    return ans;
}
```

4.14 Generating Functions

Note: OGF is for combinations (order doesn't matter), EGF is for permutations (order matters).

- OGF: $A(x) = \sum_{i \geq 0} a_i x^i$
- $x A'(x) \Rightarrow n a_n$, $A(x) B(x) \Rightarrow \sum_{i=0}^n a_i b_{n-i}$
- EGF: $A(x) = \sum_{i \geq 0} a_i \frac{x^i}{i!}$
- $A'(x) \Rightarrow a_{n+1}$, $x A(x) \Rightarrow n a_n$, $A(x) B(x) \Rightarrow \sum_{i=0}^n \binom{n}{i} a_i b_{n-i}$

5 Data Structure

5.1 PBDS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T, class Cmp = less<T>>
using OS = tree<T, null_type, Cmp, rb_tree_tag,
               tree_order_statistics_node_update>;
```

```
// Multiset: strictly use pair<T, int> to avoid UB!
// Insert: s.insert({val, id++})
// Erase exactly x: s.erase(s.lower_bound({x, 0}))
```

```
// find_by_order(k) -> iterator to k-th element
// order_of_key({x, 0}) -> count of elements < x
```

```
// Lower Bound of x (first element >= x):
// int idx = s.order_of_key({x, 0}); // 0-indexed
// if (idx == s.size()) no_lower_bound;
// else lb = s.find_by_order(idx)->first;
```

```
// Max element < x: s.find_by_order(idx - 1)->first
// Upper Bound of x: s.order_of_key({x + 1, 0})
```

```
// --- Custom Comparator Example ---
```

```
struct cmp {
    bool operator()(int a, int b) { return a > b; } //
    //   ↪ must be strict
};
using custom_set = OS<int, cmp>;
```

5.2 GP Hash Table

Usage: `gp_hash.table<int, int, custom_hash> mp;`
(Drop-in replacement for `unordered_map` to prevent anti-

hash tests).

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch()
                .count();
        return splitmix64(x + FIXED_RANDOM);
    }
    size_t operator()(
        const pair<ll, ll>& p) const noexcept {
        uint64_t a =
            splitmix64(p.first + 0x9e3779b97f4a7c15ULL);
        uint64_t b = splitmix64(p.second +
            0x9e3779b97f4a7c15ULL + 0xc6a4a7935bd1e995ULL);
        return (size_t)(a ^ (b << 1));
    }
};
gp_hash_table<pair<ll, ll>, ll, custom_hash> mp;
```

5.3 DSU

KACTL-style single-vector DSU: $e[x]$ is negative for roots (stores $-size$) and non-negative for parents. 66% memory savings vs 3-vector approach. $size(x)$ returns component size. cnt tracks number of components.

```
struct dsu {
    vector<int> e; // negative = size (root),
    ↪ non-negative = parent
    int cnt;
    dsu(int n) : e(n, -1), cnt(n) {}
    int find(int x) {
        return e[x] < 0 ? x : e[x] = find(e[x]);
    }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (e[x] > e[y]) swap(x, y); // more negative =
        ↪ larger size
        e[x] += e[y]; e[y] = x; --cnt;
        return true;
    }
    int size(int x) { return -e[find(x)]; }
};
```

5.4 Sparse Table

Range query (max/min/GCD). Idempotent — overlapping intervals OK. Build $O(N \log N)$, query $O(1)$. No updates. Use log-major layout for cache efficiency.

```
struct Sparse {
    vector<vector<int>> t;
    Sparse(const vector<int>& a) {
        int n = a.size(), lg = n ? 32 - __builtin_clz(n) :
        ↪ 0;
```

```
        t.assign(lg, vector<int>(n));
        t[0] = a;
        for (int j = 1; j < lg; j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                t[j][i] = max(t[j-1][i], t[j-1][i+(1<<(j-1))]);
    }
    int qry(int l, int r) const {
        int j = 31 - __builtin_clz(r-l+1);
        return max(t[j][l], t[j][r-(1<<j)+1]);
    }
};
// For min: s/max/min/ For gcd: s/max/gcd/
```

5.5 Sparse Table 2D

Build $O(NM \log N \log M)$, query $O(1)$. **Memory intensive:** ~ 600 MB for 2048^2 grid (11×11 matrices). Use for small grids only. Prefer 2D Fenwick for larger grids.

```
struct Sparse2D {
    int n, m;
    vector<vector<int>> a;
    vector<int> lg;
    vector<vector<int>> st[11][11]; // st[logA][logB] is
    ↪ a matrix; supports up to 2047
    Sparse2D(int n, int m) : n(n), m(m) {
        a.assign(n, vector<int>(m));
        lg.resize(max(n, m) + 1);
        for (int i = 2; i <= max(n, m); i++) lg[i] =
        ↪ lg[i>>1] + 1;
    }
    void set_val(int i, int j, int v) { a[i][j] = v; }
    void build() {
        auto &s = st[0][0];
        s.assign(n, vector<int>(m));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                s[i][j] = a[i][j];
        int la = lg[n], lb = lg[m];
        for (int ka = 0; ka <= la; ka++) {
            for (int kb = 0; kb <= lb; kb++) {
                if (ka + kb == 0) continue;
                int pwa = 1 << ka, pwb = 1 << kb;
                auto &cur = st[ka][kb];
                cur.assign(n - pwa + 1, vector<int>(m - pwb +
                ↪ 1));
                for (int i = 0; i + pwa <= n; i++)
                    for (int j = 0; j + pwb <= m; j++) {
                        if (!ka)
                            cur[i][j] = max(st[ka][kb-1][i][j],
                                st[ka][kb-1][i][j+pwb/2]);
                        else
                            cur[i][j] = max(st[ka-1][kb][i][j],
                                st[ka-1][kb][i+pwa/2][j]);
                    }
            }
        }
    }
    int qry(int x1, int y1, int x2, int y2) const {
        int a = lg[x2-x1+1], b = lg[y2-y1+1];
```

```
        auto &s = st[a][b];
        int m1 = max(s[x1][y1], s[x2-(1<<a)+1][y1]);
        int m2 = max(s[x1][y2-(1<<b)+1],
            s[x2-(1<<a)+1][y2-(1<<b)+1]);
        return max(m1, m2);
    }
};
5.6 Fenwick Tree
Point update, prefix/range sum.  $O(\log N)$  per op. 1-indexed
internally. Usage: BIT<ll> fenw(n); fenw.add(i, val);
fenw.sum(l, r);
template<typename T>
struct BIT {
    int n;
    vector<T> bit;
    BIT(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, 0); }
    void add(int i, T v) {
        // walk up: i += i & -i moves to next responsible
        ↪ ancestor
        for (; i <= n; i += i & -i) bit[i] += v; }
    T sum(int i) {
        T r = 0;
        // walk down: i -= i & -i strips lowest set bit
        for (; i > 0; i -= i & -i) r += bit[i];
        return r;
    }
    T sum(int l, int r) { return sum(r) - sum(l - 1); }
    // find smallest i with prefix_sum(i) >= k (values
    ↪ must be non-negative)
    int find_kth(T k) {
        int i = 0;
        for (int j = 1 << (31 - __builtin_clz(n)); j; j
        ↪ >>= 1)
            if (i + j <= n && bit[i + j] < k) k -= bit[i +=
            ↪ j];
        return i + 1;
    }
};
```

Usage: BIT<ll> fenw(5); fenw.find_kth(k) returns 1-indexed position.

5.7 Fenwick 2D

```
struct BIT2D {
    int n, m;
    vector<vector<ll>> bit;
    BIT2D(int n, int m) : n(n), m(m) {
        bit.assign(n + 1, vector<ll>(m + 1, 0));
    }
    void add(int x, int y, ll val) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j)
                bit[i][j] += val;
    }
    ll sum(int x, int y) {
        ll res = 0;
```

```

for (int i = x; i > 0; i -= i & -i)
    for (int j = y; j > 0; j -= j & -j)
        res += bit[i][j];
return res;
}
ll query(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - sum(x1 - 1, y2) -
        sum(x2, y1 - 1) + sum(x1 - 1, y1 - 1);
}
};
BIT2D fenw2(4, 4);
fenw2.add(1, 1, 5);
fenw2.add(3, 2, 7);
cout << fenw2.query(1, 1, 3, 3) << "\n";

```

5.8 BIT Range Update & Range Query

Uses 2 BITS trick. Add val to $[l, r]$, query sum of $[l, r]$. Derive from prefix sum identity: $psum(i) = i \cdot bit_1(i) - bit_2(i)$.

```

struct BITRange {
    int n;
    vector<ll> bit1, bit2;
    BITRange(int n) : n(n), bit1(n + 2), bit2(n + 2) {}
    void add(vector<ll>& bit, int i, ll v) {
        for (; i <= n; i += i & -i) bit[i] += v;
    }
    void upd(int l, int r, ll v) {
        add(bit1, l, v);
        add(bit1, r + 1, -v);
        add(bit2, l, v * (l - 1));
        add(bit2, r + 1, -(r * v));
    }
    ll sum(vector<ll>& bit, int i) {
        ll r = 0;
        for (; i > 0; i -= i & -i) r += bit[i];
        return r;
    }
    ll qry(int l, int r) {
        auto pref = [&](int i) { return sum(bit1, i) * i -
            ⇨ sum(bit2, i); };
        return pref(r) - pref(l - 1);
    }
};
BITRange bit(n);    bit.upd(l, r, v);    bit.qry(l, r);

```

5.9 Lazy Segment Tree

Range add/set, range sum/max/min. $O(\log N)$ per op. Uses NO_SET sentinel to eliminate the hz flag vector – saves $\sim 2.4\%$ memory. Requires using `ll = long long`; in setup.

```

struct LSeg {
    int n;
    vector<ll> sm, mx, mn, ad, st;
    const ll INF = 1e18, NO_SET = 9e18;
    LSeg(int n) : n(n), sm(4*n), mx(4*n), mn(4*n),
        ad(4*n), st(4*n, NO_SET) {}
    void pull(int i) {
        sm[i] = sm[i*2] + sm[i*2+1];
        mx[i] = max(mx[i*2], mx[i*2+1]);
        mn[i] = min(mn[i*2], mn[i*2+1]);
    }
};

```

```

}
void pset(int i, int s, int e, ll v) {
    st[i] = v; ad[i] = 0;
    sm[i] = v * (e - s + 1); mx[i] = mn[i] = v;
}
void padd(int i, int s, int e, ll v) {
    if (st[i] != NO_SET) st[i] += v; else ad[i] += v;
    sm[i] += v * (e - s + 1); mx[i] += v; mn[i] += v;
}
void push(int i, int s, int e) {
    if (s == e) return;
    int m = (s + e) / 2;
    if (st[i] != NO_SET) { pset(i*2, s, m, st[i]);
        pset(i*2+1, m+1, e, st[i]); st[i] = NO_SET; }
    if (ad[i]) { padd(i*2, s, m, ad[i]);
        padd(i*2+1, m+1, e, ad[i]); ad[i] = 0; }
}
void upd(int l, int r, ll v, int t,
    int i=1, int s=0, int e=-1) {
    if (e < 0) e = n-1;
    if (r < s || e < l) return;
    if (l <= s && e <= r)
        { t? pset(i, s, e, v) : padd(i, s, e, v); return; }
    push(i, s, e); int m = (s + e) / 2;
    upd(l, r, v, t, i*2, s, m);
    upd(l, r, v, t, i*2+1, m+1, e);
    pull(i);
}
array<ll, 3> qry(int l, int r,
    int i=1, int s=0, int e=-1) {
    if (e < 0) e = n-1;
    if (r < s || e < l) return {0, -INF, INF};
    if (l <= s && e <= r) return {sm[i], mx[i], mn[i]};
    push(i, s, e); int m = (s + e) / 2;
    auto a = qry(l, r, i*2, s, m),
        b = qry(l, r, i*2+1, m+1, e);
    return {a[0] + b[0], max(a[1], b[1]),
        min(a[2], b[2])};
}
void set(int l, int r, ll v) { upd(l, r, v, 1); }
void add(int l, int r, ll v) { upd(l, r, v, 0); }
// auto [s, mx, mn] = seg.qry(l, r);
// seg.set(l, r, v); seg.add(l, r, v);
};

```

5.10 Persistent Segment Tree

Persistent segment tree supporting k-th smallest, count $\leq x$, and range sum (distinct elements). All operations $O(\log N)$. Each version = new root.

Usage 1 – k-th smallest / count $\leq x$ (value tree):

```

root[0] = build(1, m);    (m = #unique values)
for i=1..n:    root[i] = upd(root[i-1], 1, m,
    comp[a[i]], +1)
kth(root[l-1], root[r], 1, m, k)    k-th smallest in [l, r]
cnt(root[l-1], root[r], 1, m, x)    # ≤ x in [l, r]
cnt(root[l-1], root[r], y) - cnt(x-1)    # ∈ [x, y]

```

Usage 2 – distinct count (position tree):

```

root[0] = build(1, n);
for i=1..n:
    r = root[i-1];
    if (last[a[i]]) r = upd(r, 1, n, last[a[i]], -1);
    root[i] = upd(r, 1, n, i, +1);
    last[a[i]] = i;
sum(root[0], root[r], 1, n, l, r)    distinct count in
[l, r]
struct PST {
    static const int N = 2'00'000;
    static const int LG = 18;
    static const int M = 1 + 4 * N + N * LG;
    int lc[M], rc[M], val[M];
    int sz = 1;

    int build(int b, int e) {
        int cur = sz++;
        if (b == e) return cur;
        int m = (b + e) >> 1;
        lc[cur] = build(b, m);
        rc[cur] = build(m + 1, e);
        return cur;
    }
    int upd(int pre, int b, int e, int i, int v) {
        int cur = sz++;
        lc[cur] = lc[pre]; rc[cur] = rc[pre];
        if (b == e) { val[cur] = val[pre] + v; return cur;
            ⇨ }
        int m = (b + e) >> 1;
        if (i <= m) lc[cur] = upd(lc[pre], b, m, i, v);
        else rc[cur] = upd(rc[pre], m + 1, e, i, v);
        val[cur] = val[lc[cur]] + val[rc[cur]];
        return cur;
    }
    int kth(int pre, int cur, int b, int e, int k) {
        while (b < e) {
            int m = (b + e) >> 1;
            int left_cnt = val[lc[cur]] - val[lc[pre]];
            if (k <= left_cnt) { cur = lc[cur]; pre =
                ⇨ lc[pre]; e = m; }
            else { cur = rc[cur]; pre =
                ⇨ rc[pre]; b = m + 1; k -= left_cnt; }
        }
        return b;
    }
}
int cnt(int pre, int cur, int b, int e, int x) {
    if (x < b) return 0;
    int res = 0;
    while (b < e) {
        int m = (b + e) >> 1;
        if (x <= m) { cur = lc[cur]; pre = lc[pre]; e =
            ⇨ m; }
        else { res += val[lc[cur]] - val[lc[pre]]; cur =
            ⇨ rc[cur]; pre = rc[pre]; b = m + 1; }
    }
}

```

```

}
res += val[cur] - val[pre];
return res;
}
int sum(int pre, int cur, int b, int e, int l, int r)
↪ {
    if (l > e || r < b) return 0;
    if (l <= b && e <= r) return val[cur] - val[pre];
    int m = (b + e) >> 1;
    return sum(lc[pre], lc[cur], b, m, l, r) +
    ↪ sum(rc[pre], rc[cur], m + 1, e, l, r);
}
};

```

5.11 Merge Sort Tree

Segment tree where each node stores a sorted list of its range. $qry(l, r, k)$ = number of elements $\leq k$ in $[l, r]$. Build $O(N \log N)$, query $O(\log^2 N)$, space $O(N \log N)$.

```

struct MergeSortTree {
    int n;
    vector<vector<int>> tree;
    MergeSortTree(const vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(1, 0, n - 1, arr);
    }
    void build(int node, int l, int r,
               const vector<int>& arr) {
        if (l == r) {
            tree[node].push_back(arr[l]);
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid, arr);
        build(2 * node + 1, mid + 1, r, arr);
        merge(tree[2 * node].begin(), tree[2 * node].end(),
              tree[2 * node + 1].begin(),
              tree[2 * node + 1].end(),
              back_inserter(tree[node]));
    }
    int query(int node, int l, int r, int ql, int qr, int
    ↪ k) {
        if (l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) {
            // binary search in sorted node list: count <= k
            return upper_bound(tree[node].begin(),
                               tree[node].end(), k) - tree[node].begin();
        }
        int mid = (l + r) / 2;
        return query(2 * node, l, mid, ql, qr, k) +
        ↪ query(2 * node + 1, mid + 1, r, ql, qr, k);
    }
};

```

5.12 Wavelet Matrix

Wavelet Matrix (compact iterative WT) using `vector<uint64_t>` bit vectors instead of `vector<int>` prefix arrays — $32\times$ memory savings per level. Build $O(N \log M)$,

query $O(\log M)$. 0-indexed. $kth(l, r, k)$ returns k -th smallest (0-based) in $[l, r]$. $count_leq(l, r, x)$ counts $\leq x$. $count_seg(l, r, x, y)$ counts in $[x, y]$.

```

struct WT {
    int n, lg;
    vector<vector<uint64_t>> b;
    vector<vector<int>> p; // prefix sum over 64-bit
    ↪ blocks
    vector<int> z; // number of 0s at each level
    WT(vector<int> v) : n(v.size()) {
        int mx = n ? *max_element(v.begin(), v.end()) : 0;
        lg = mx ? 32 - __builtin_clz(mx) : 0;
        if (!lg) { b.resize(1); p.resize(1); z.resize(1);
        ↪ return; }
        b.resize(lg); p.resize(lg); z.resize(lg);
        vector<int> cur = v, nxt(n);
        for (int d = lg - 1; d >= 0; --d) {
            int bc = (n + 63) / 64;
            b[d].assign(bc, 0);
            int p0 = 0, p1 = 0;
            for (int i = 0; i < n; ++i) {
                if (cur[i] >> d & 1) {
                    b[d][i / 64] |= 1ULL << (i % 64);
                    nxt[p1++] = cur[i];
                } else cur[p0++] = cur[i];
            }
            z[d] = p0;
            copy(nxt.begin(), nxt.begin() + p1, cur.begin()
            ↪ + p0);
            p[d].resize(bc + 1);
            for (int i = 0; i < bc; ++i)
                p[d][i + 1] = p[d][i] +
                ↪ __builtin_popcountll(b[d][i]);
        }
    }
    int rank1(int d, int i) const {
        int bc = (n + 63) / 64;
        if (i >= n) return p[d][bc];
        int r = i / 64;
        return p[d][r] + __builtin_popcountll(
        ↪ b[d][r] & ((1ULL << (i % 64)) - 1));
    }
    int rank0(int d, int i) const { return i - rank1(d,
    ↪ i); }
    int kth(int l, int r, int k) {
        int ans = 0;
        for (int d = lg - 1; d >= 0; --d) {
            int c0 = rank0(d, r) - rank0(d, l);
            if (k < c0) {
                l = rank0(d, l); r = rank0(d, r);
            } else {
                ans |= 1 << d; k -= c0;
                l = z[d] + rank1(d, l); r = z[d] + rank1(d, r);
            }
        }
        return ans;
    }
};

```

```

int count_leq(int l, int r, int x) {
    if (x < 0) return 0;
    if (r <= 1) return 0;
    if (x >= (1LL << lg)) return r - 1;
    int ans = 0;
    for (int d = lg - 1; d >= 0; --d) {
        int c0 = rank0(d, r) - rank0(d, l);
        if (x >> d & 1) {
            ans += c0;
            l = z[d] + rank1(d, l); r = z[d] + rank1(d, r);
        } else {
            l = rank0(d, l); r = rank0(d, r);
        }
    }
    return ans + (r - 1);
}
int count_seg(int l, int r, int x, int y) {
    if (x > y) return 0;
    return count_leq(l, r, y) - count_leq(l, r, x - 1);
}
};

```

5.13 Trie (Binary)

Binary Trie for integers up to $2^B - 1$. $O(B)$ per op. Uses single `vector<Node>` pool for cache locality (all node data in one cache line vs two separate vectors). $add(x, v)$ – insert ($v = 1$) or erase ($v = -1$)

$count(x)$ – how many times x inserted

$count_xor_less(x, k) - \#\{y : (x \oplus y) < k\}$

$count_xor_greater(x, k) - \#\{y : (x \oplus y) > k\}$

$get_kth(k)$ – k -th smallest value (1-indexed)

$get_xor_kth(x, k)$ – k -th smallest $(x \oplus y)$ over y

$count_special(x) - \#\{y : y \leq (x \oplus y) \leq x\}$

```

struct Trie {
    static const int B = 31;
    struct Node { int nxt[2]; int cnt; };
    vector<Node> pool;
    Trie() { pool.push_back({0, 0, 0}); }
    void add(int x, int v = 1) {
        int u = 0; pool[u].cnt += v;
        for (int i = B - 1; i >= 0; --i) {
            int d = (x >> i) & 1;
            if (!pool[u].nxt[d]) {
                pool[u].nxt[d] = pool.size();
                pool.push_back({0, 0, 0});
            }
            u = pool[u].nxt[d]; pool[u].cnt += v;
        }
    }
    int count(int x) {
        int u = 0;
        for (int i = B - 1; i >= 0; --i) {
            int d = (x >> i) & 1;
            int nxt = pool[u].nxt[d];
            if (!nxt || !pool[nxt].cnt) return 0;
            u = nxt;
        }
    }
};

```

```

}
return pool[u].cnt;
}

int count_xor_less(int x, int k) {
    int u = 0, ans = 0;
    for (int i = B - 1; i >= 0; --i) {
        int d = (x >> i) & 1;
        if ((k >> i) & 1) {
            if (pool[u].nxt[d]) ans +=
                pool[pool[u].nxt[d]].cnt;
            if (!pool[u].nxt[d ^ 1]) break;
            u = pool[u].nxt[d ^ 1];
        } else {
            if (!pool[u].nxt[d]) break;
            u = pool[u].nxt[d];
        }
    }
    return ans;
}

int count_xor_greater(int x, int k) {
    if (k >= (1LL << B) - 1) return 0;
    return pool[0].cnt - count_xor_less(x, k + 1);
}

int get_kth(int k) { if (pool[0].cnt < k) return -1;
    int u = 0, ans = 0;
    for (int i = B - 1; i >= 0; --i) {
        int left = pool[u].nxt[0] ?
            pool[pool[u].nxt[0]].cnt : 0;
        if (k <= left) u = pool[u].nxt[0];
        else k -= left, u = pool[u].nxt[1], ans |= (1 <<
            i);
    }
    return ans;
}

int get_xor_kth(int x, int k) { if (pool[0].cnt < k)
    ↪ return -1;
    int u = 0, ans = 0;
    for (int i = B - 1; i >= 0; --i) {
        int d = (x >> i) & 1;
        int req = pool[u].nxt[d] ?
            pool[pool[u].nxt[d]].cnt : 0;
        if (k <= req)
            u = pool[u].nxt[d];
        else k -= req, u = pool[u].nxt[d ^ 1],
            ans |= (1 << i);
    }
    return ans;
}

int64_t go(int u, int b, int x, int t1, int t2) {
    if (!t1 || t2 == 2) return 0;
    if ((t1 == 2 && !t2) || b < 0) return pool[u].cnt;
    int d = (x >> b) & 1; int64_t res = 0;
    if (d) {
        if (pool[u].nxt[1]) res += go(pool[u].nxt[1], b
            ↪ - 1, x,
                t1 == 1 ? 0 : t1, t2 == 1 ? 0 : t2);
        if (pool[u].nxt[0]) res += go(pool[u].nxt[0], b
            ↪ - 1, x,
                t1 == 1 ? 2 : t1, t2);
    }
}

```

```

} else {
    if (pool[u].nxt[1]) res += go(pool[u].nxt[1], b
        ↪ - 1, x, t1,
            t2 == 1 ? 2 : t2);
    if (pool[u].nxt[0]) res += go(pool[u].nxt[0], b
        ↪ - 1, x, t1, t2);
}
return res;
}

int64_t count_special(int x) {
    return go(0, B - 1, x, 1, 1);
}
};

```

5.14 Mo's Algorithm

add(pos) and rem(pos) must be implemented per problem. Snake-order (even/odd block sorting) for $O(N\sqrt{Q})$. Sort queries first. Block size $B = \sqrt{N}$. Alternating sort order within even/odd blocks cuts the total right-pointer moves nearly in half in practice.

// sort queries: blocks of size B, snake-order within
 ↪ *blocks*

```

sort(cu.begin(), cu.end(), [&](auto &x, auto &y){
    if (x.l / B != y.l / B)
        return x.l / B < y.l / B;
    return (x.l / B) % 2 ? x.r > y.r : x.r < y.r;
});

```

```

int R = -1, L = 0; // [L,R] is current active window
while (R < r) add(++R); // expand right first
while (L > l) add(--L); // expand left
while (R > r) rem(R--); // shrink right
while (L < l) rem(L++); // shrink left

```

5.15 Rollback DSU

DSU with undo. No path compression — union by size only. time() returns a snapshot marker; rollback(t) reverts to that state. **When to use:** offline dynamic connectivity, DFS with backtracking, problems requiring DSU with rollback.

```

struct RollbackDSU {
    vector<int> e;
    vector<pair<int,int>> hist;
    RollbackDSU(int n) : e(n, -1) {}
    int find(int x) {
        while (e[x] >= 0) x = e[x];
        return x;
    }
    int size(int x) { return -e[find(x)]; }
    int time() { return hist.size(); }
    void rollback(int t) {
        while ((int)hist.size() > t) {
            auto [x, v] = hist.back(); hist.pop_back();
            e[x] = v;
        }
    }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        hist.push_back({a, e[a]});
    }
};

```

```

        hist.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};

```

5.16 Disjoint Sparse Table

Non-idempotent $O(1)$ range query (sum, xor, AND, OR). Build $O(N \log N)$, query $O(1)$. No updates. Pads to power of 2 internally. **When to use:** range sum/xor/AND queries with $O(1)$ time, complement to regular sparse table.

```

struct DST {
    int n, log;
    vector<vector<int>> t;
    DST(vector<int> a) : n(a.size()) {
        if (n == 0) { log = 0; return; }
        int m = 1;
        while (m < n) m <= 1;
        a.resize(m, 0);
        log = 32 - __builtin_clz(m);
        t.assign(log, vector<int>(m));
        t[0] = a;
        for (int h = 1; h < log; h++) {
            int half = 1 << (h - 1);
            for (int i = 0; i < m; i += half * 2) {
                int mid = i + half;
                t[h][mid - 1] = a[mid - 1];
                for (int j = mid - 2; j >= i; j--)
                    t[h][j] = t[h][j + 1] + a[j];
                t[h][mid] = a[mid];
                for (int j = mid + 1; j < i + half * 2; j++)
                    t[h][j] = t[h][j - 1] + a[j];
            }
        }
    }
    int query(int l, int r) {
        if (l == r) return t[0][l];
        int h = 32 - __builtin_clz(l ^ r);
        return t[h][l] + t[h][r];
    }
};

```

5.17 Min Queue

Sliding window minimum in $O(1)$ amortized per op. Maintains deque with increasing values. Works for max by changing the comparison. **When to use:** sliding window min/max, monotonic queue problems, subsequence constraints.

```

struct MinQueue {
    deque<int> dq; // increasing values
    queue<int> q; // actual elements
    void push(int x) {
        q.push(x);
        while (!dq.empty() && dq.back() > x) dq.pop_back();
        dq.push_back(x);
    }
};

```

```
void pop() {
    if (q.front() == dq.front()) dq.pop_front();
    q.pop();
}
int get_min() { return dq.front(); }
bool empty() { return q.empty(); }
};
// For max: change > to < in push, rename to MaxQueue.
```

5.18 Small-to-Large Merging

Generic technique: when merging two sets, always merge the smaller into the larger. Total cost $O(N \log N)$ across all merges. **When to use:** maintaining per-component data during DSU union or tree DP. **Memory tip:** Use `vector<int>` instead of `set<int>` when duplicates are not needed — `vector` saves $7-8\times$ memory (4 bytes vs ~ 32 bytes for `set` tree nodes). Sort + unique after merge if needed.

// Example: DSU + vector per component (use vector not $\hookrightarrow$ set)

`vector<vector<int>>` comp(n); *// vector: $\sim 8\times$ less memory $\hookrightarrow$ than set*

```
vector<int> par(n);
iota(par.begin(), par.end(), 0);
function<int(int)> find = [&](int x) {
    while (par[x] != x) x = par[x] = par[par[x]];
    return x;
};
```

```
auto unite = [&](int u, int v) {
    u = find(u), v = find(v);
    if (u == v) return;
    if (comp[u].size() < comp[v].size()) swap(u, v);
    par[v] = u;
    comp[u].insert(comp[u].end(),
        comp[v].begin(), comp[v].end());
    comp[v].clear();
    // sort(comp[u].begin(), comp[u].end());
    // comp[u].erase(unique(...), comp[u].end());
};
```

6 Strings

6.1 String Double Hashing

Static $O(1)$ substring hash, two moduli. `DHash(s)` builds prefix hashes. `get(l,r)` returns `pair<ll,ll>`. Compare via `get(l1,r1)==get(l2,r2)`.

```
struct DHash {
    ll mod[2]={1000000007,1000000009},
    base[2]={911382323,972663749};
    vector<ll> pw[2],h[2];
    DHash(string &s) {
        int n=s.size();
        for(int k=0;k<2;k++) {
            pw[k].resize(n+1,1); h[k].resize(n+1);
            for(int i=0;i<n;i++) {
                pw[k][i+1]=pw[k][i]*base[k]%mod[k];
                h[k][i+1]=(h[k][i]*base[k]+s[i])%mod[k];
            }
        }
    }
};
```

```
}
pair<ll,ll> get(int l,int r) {
    return {
        (h[0][r+1]-h[0][l]*pw[0][r-l+1]%mod[0]
            +mod[0])%mod[0],
        (h[1][r+1]-h[1][l]*pw[1][r-l+1]%mod[1]
            +mod[1])%mod[1],
    };
}
```

6.2 Dynamic String Hash

Segment tree with point update ($O(\log N)$), forward/reverse hash, palindrome check. 1-indexed. Set N to max length. Call `init_hash(N)` once.

```
const int MOD[]={127657753,987654319},
    BASE[]={137,277};
int pw[2][N],ipw[2][N];
int mpow(int a,int n,int m) {
    int r=1;
    while(n) { if(n&1) r=1LL*r*a%m; a=1LL*a*a%m; n>>=1; }
    return r;
}
```

```
void init_hash(int n) {
    for(int k=0;k<2;k++) {
        pw[k][0]=ipw[k][0]=1;
        for(int i=1;i<=n;i++)
            pw[k][i]=1LL*pw[k][i-1]*BASE[k]%MOD[k];
        int inv=mpow(BASE[k],MOD[k]-2,MOD[k]);
        for(int i=1;i<=n;i++)
            ipw[k][i]=1LL*ipw[k][i-1]*inv%MOD[k];
    }
}
```

```
using H = array<int,2>;
H add(H a,H b) { return
    {(a[0]+b[0])%MOD[0],(a[1]+b[1])%MOD[1]}; }
H mul(H a,H b) { return {(int)(1LL*a[0]*b[0]%MOD[0]),
    (int)(1LL*a[1]*b[1]%MOD[1])}; }
}
```

```
struct SegHash {
    int n;
    vector<H> fwd,rev;
    SegHash(string s) {
        n=s.size(); s="."+s;
        fwd.resize(4*n); rev.resize(4*n);
        build(1,1,n,s);
    }
    void build(int node,int l,int r,string &s) {
        if(l==r) {
            fwd[node]=
                {(int)(1LL*s[l]*pw[0][1]%MOD[0]),
                 (int)(1LL*s[l]*pw[1][1]%MOD[1])};
            rev[node]=
                {(int)(1LL*s[l]*pw[0][n-l+1]%MOD[0]),
                 (int)(1LL*s[l]*pw[1][n-l+1]%MOD[1])};
            return;
        }
        int mid=(l+r)/2;
```

```
        build(node*2,l,mid,s);
        build(node*2+1,mid+1,r,s);
        fwd[node]=add(fwd[node*2],fwd[node*2+1]);
        rev[node]=add(rev[node*2],rev[node*2+1]);
    }
    void upd(int i,char c) { upd(1,1,n,i,c); }
    void upd(int node,int l,int r,int idx,char c) {
        if(l==r) {
            fwd[node]=
                {(int)(1LL*c*pw[0][1]%MOD[0]),
                 (int)(1LL*c*pw[1][1]%MOD[1])};
            rev[node]=
                {(int)(1LL*c*pw[0][n-l+1]%MOD[0]),
                 (int)(1LL*c*pw[1][n-l+1]%MOD[1])};
            return;
        }
        int mid=(l+r)/2;
        if(idx<=mid) upd(node*2,l,mid,idx,c);
        else upd(node*2+1,mid+1,r,idx,c);
        fwd[node]=add(fwd[node*2],fwd[node*2+1]);
        rev[node]=add(rev[node*2],rev[node*2+1]);
    }
    H qry(vector<H>& t,int node,int l,int r,int i,int j)
     $\hookrightarrow$  {
        if(i>r||j<l) return {0,0};
        if(i<=l&&r<=j) return t[node];
        int mid=(l+r)/2;
        return add(qry(t,node*2,l,mid,i,j),
            qry(t,node*2+1,mid+1,r,i,j));
    }
    H get_hash(int l,int r) {
        H h = qry(fwd,1,1,n,l,r);
        return mul(h,{ipw[0][l-1],ipw[1][l-1]});
    }
    H get_rev(int l,int r) {
        H h = qry(rev,1,1,n,l,r);
        return mul(h,{ipw[0][n-r],ipw[1][n-r]});
    }
    bool is_pal(int l,int r) {
        return get_hash(l,r)==get_rev(l,r);
    }
};
```

6.3 LPS and SPS

$O(N)$. `getLps(pattern)` returns LPS (longest proper prefix which is also suffix). `getSps(pi)` returns SPS (smallest proper prefix which is also suffix).

```
vector<int> getLps(const string &t) {
    int n = t.size();
    vector<int> pi(n, 0);
    int lps = 0;
    for (int q = 1; q < n; q++) {
        while (lps > 0 && t[lps] != t[q])
            lps = pi[lps - 1];
        if (t[lps] == t[q]) lps += 1;
        pi[q] = lps;
    }
}
```

```

    return pi;
}
vector<int> getSps(const vector<int> &pi) {
    int n = pi.size();
    vector<int> sps(n, 0);
    for (int i = 1; i < n; i++) {
        if (pi[i] == 0) sps[i] = 0;
        else if (pi[pi[i] - 1] == 0) sps[i] = pi[i];
        else sps[i] = sps[pi[i] - 1];
    }
    return sps;
}

```

Usage: auto pi = getLps(s); auto sp = getSps(pi);

6.4 Z-Algorithm

$O(N)$. $z[i]$ = longest common prefix of $s[0..n-1]$ and $s[i..n-1]$.

```

vector<int> getZ(const string &s) {
    int n = s.size();
    if (n == 0) return {};
    vector<int> z(n);
    z[0] = 0;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}

```

6.5 Manacher's Algorithm

$O(N)$. $p[1][i]$ = odd palindrome radius at i , $p[0][i]$ = even palindrome radius (right center).

```

struct Manacher {
    vector<int> p[2];
    Manacher(string s) {
        int n = s.size();
        p[0].resize(n + 1); // even-len palindromes (right
        ↪ center)
        p[1].resize(n); // odd-len palindromes (center
        ↪ at i)
        for (int z = 0; z < 2; z++) {
            for (int i = 0, l = 0, r = 0; i < n; i++) {
                int t = r - i + !z;
                if (i < r) p[z][i] = min(t, p[z][l + t]);
                int L = i - p[z][i], R = i + p[z][i] - !z;
                while (L >= 1 && R + 1 < n &&
                    s[L - 1] == s[R + 1])
                    p[z][i]++, L--, R++;
                if (R > r) l = L, r = R; // update rightmost
                ↪ pal
            }
        }
    }
    bool is_pal(int l, int r) {

```

```

        int mid = (l + r + 1) / 2, len = r - l + 1;
        return 2 * p[len % 2][mid] + len % 2 >= len;
    }
};

```

6.6 Suffix Array

$O(N \log N)$ prefix-doubling. Returns SA and LCP. $sa[i]$ = starting index of i -th smallest suffix. $lcp[i]$ = LCP of $sa[i]$ and $sa[i-1]$ ($lcp[0]=0$). **Tricks:** Distinct substrings = $\frac{n(n+1)}{2} - \sum lcp[i]$. Longest repeated substring = $\max(lcp)$. Works with string or vector<int>. lim = alphabet size (128 for ASCII).

```

array<vector<int>, 2> get_sa(auto& s, int lim = 128) {
    int n = s.size() + 1, k = 0, a, b;
    vector<int> x(begin(s), end(s));
    x.push_back(0);
    vector<int> sv = x, y(n), sa(n), lcp(n), ws(max(n,
    ↪ lim)), rank(n);
    iota(begin(sa), end(sa), 0);
    for (int j = 0, p = 0; p < n;
        j = max(1, j * 2), lim = p) {
        p = j, iota(begin(y), end(y), n - j);
        for (int i = 0; i < n; i++)
            if (sa[i] >= j) y[p++] = sa[i] - j;
        fill(begin(ws), end(ws), 0);
        for (int i = 0; i < n; i++) ws[x[i]]++;
        for (int i = 1; i < lim; i++) ws[i] += ws[i - 1];
        for (int i = n; i--;)
            sa[--ws[x[y[i]]]] = y[i];
        swap(x, y), p = 1, x[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            a = sa[i - 1], b = sa[i],
            x[b] = (y[a] == y[b] && y[a + j] == y[b + j])
                ? p - 1 : p++;
    }
    for (int i = 1; i < n; i++) rank[sa[i]] = i;
    for (int i = 0, j; i < n - 1; lcp[rank[i+]] = k)
        for (k && k--, j = sa[rank[i] - 1];
            sv[i + k] == sv[j + k]; k++);
    sa.erase(sa.begin()), lcp.erase(lcp.begin());
    return {sa, lcp};
}

```

6.7 Aho-Corasick

Multiple pattern matching in $O(N + \sum |P| + occ)$. $ins(s)$ inserts pattern, $build_fails()$ builds fail + output links. Traverse text via $trie[cur][c]$, follow $exit_link[cur]$ for matches. $is_end[cur]$ = length of pattern ending at node (0 if none).

```

const int N = 1e6 + 6;
int trie[N][26], fail[N], exit_link[N], is_end[N];
int ptr;
void init() {
    for (int i = 0; i <= ptr; i++) {
        fill(trie[i], trie[i] + 26, 0);
        fail[i] = exit_link[i] = is_end[i] = 0;
    }
}

```

```

ptr = 1;
}
void ins(string &s) {
    int cur = 0;
    for (auto c : s) {
        int d = c - 'a';
        if (!trie[cur][d]) trie[cur][d] = ptr++;
        cur = trie[cur][d];
    }
    is_end[cur] = s.size();
}
void build_fails() {
    queue<int> q;
    for (int i = 0; i < 26; i++)
        if (trie[0][i]) q.push(trie[0][i]);
    while (q.size()) {
        int cur = q.front(); q.pop();
        exit_link[cur] = is_end[fail[cur]] > 0
            ? fail[cur] :
            ↪ exit_link[fail[cur]];
        for (int d = 0; d < 26; d++) {
            if (trie[cur][d]) {
                q.push(trie[cur][d]);
                fail[trie[cur][d]] = trie[fail[cur]][d];
            } else {
                trie[cur][d] = trie[fail[cur]][d];
            }
        }
    }
}

```

6.8 Booth's Algorithm (Min Lex Rotation)

$O(N)$. Finds min lexicographic rotation. **Returns:** The 0-indexed starting position of the lexicographically smallest rotation.

```

int minLex(string &s) {
    int len = s.size();
    int n = 2 * len, i = 0, j = 1, k = 0;
    int a, b;
    while (i + k < n && j + k < n) {
        a = (i + k >= len) ? s[i + k - len] : s[i + k];
        b = (j + k >= len) ? s[j + k - len] : s[j + k];
        if (a == b) ++k;
        else if (a > b) {
            i = i + k + 1;
            if (i <= j) i = j + 1;
            k = 0;
        } else {
            j = j + k + 1;
            if (j <= i) j = i + 1;
            k = 0;
        }
    }
    return min(i, j);
}

```

6.9 Periods

A string has period p if $s[i] = s[i + p]$ for all i . Equivalently, $n - p$ is a border length. The chain $\pi[n - 1] \rightarrow \pi[\pi[n - 1] - 1] \rightarrow \dots \rightarrow 0$ yields all borders; subtracting from n gives all periods. **When to use:** periodicity/compression problems, checking if a string repeats, shift/cut operations.

```
vector<int> getPeriods(const vector<int> &pi) {
```

```
    if (pi.empty()) return {};
    int n = pi.size(), p = pi.back();
    vector<int> periods;
    while (p > 0) {
        periods.push_back(n - p);
        p = pi[p - 1];
    }
    return periods;
}
```

6.10 KMP Automaton

Precomputes $\text{nxt}[i][c] = \text{LPS}$ after adding character c from state i (matched prefix length). Eliminates back-tracking, enabling $O(N)$ DP over text. **When to use:** counting strings of length N that avoid a pattern, DP where pattern matching is a constraint.

```
const int K = 26;
vector<array<int,K>> buildAuto(const string &s) {
    auto pi = getLps(s);
    int n = s.size();
    vector<array<int,K>> nxt(n + 1);
    nxt[0][s[0] - 'a'] = 1;
    for (int i = 1; i <= n; i++)
        for (int c = 0; c < K; c++) {
            if (i == n || c != s[i] - 'a')
                nxt[i][c] = nxt[pi[i - 1]][c];
            else
                nxt[i][c] = i + 1;
        }
    return nxt;
}
```

6.11 Lyndon Factorization (Duval)

$O(N)$. Decomposes string into non-increasing Lyndon words. A Lyndon word is strictly smaller than all its proper rotations. **When to use:** necklace problems, string decomposition into Lyndon words, minimal string rotation (alternative to Booth).

```
vector<string> lyndon(const string &s) {
    int n = s.size();
    vector<string> res;
    for (int i = 0; i < n; ) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) {
```

```
            res.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return res;
}
```

6.12 Eertree (Palindromic Tree)

$O(N)$. Every distinct palindrome corresponds to a unique node. $\text{len}[v] = \text{palindrome length}$, $\text{link}[v] = \text{longest proper palindromic suffix}$, $\text{cnt}[v] = \text{occurrence count}$ (filled after construction). **When to use:** count distinct palindromes, palindrome occurrences, palindrome factorization, DP on palindromes.

```
struct Eertree {
    vector<array<int,26>> nxt;
    vector<int> len, link, cnt;
    int last;
    Eertree(const string &s) : last(1) {
        len = {-1, 0}; link = {0, 0}; cnt = {0, 0};
        nxt.push_back({}); nxt.push_back({});
        for (int i = 0; i < (int)s.size(); i++) add(s, i);
        for (int i = (int)len.size() - 1; i > 1; i--)
            cnt[link[i]] += cnt[i];
    }
    void add(const string &s, int i) {
        int c = s[i] - 'a', cur = last;
        while (true) {
            int cl = len[cur];
            if (i - cl - 1 >= 0 && s[i - cl - 1] == s[i])
                cur = link[cur];
        }
        if (nxt[cur][c]) {
            last = nxt[cur][c]; cnt[last]++; return;
        }
        int node = len.size();
        nxt.push_back({});
        len.push_back(len[cur] + 2);
        cnt.push_back(1);
        nxt[cur][c] = node;
        if (len[node] == 1) {
            link.push_back(1); last = node; return;
        }
        int l = link[cur];
        while (true) {
            int ll = len[l];
            if (i - ll - 1 >= 0 && s[i - ll - 1] == s[i])
                l = link[l];
        }
        link.push_back(nxt[l][c]);
        last = node;
    }
};
```

6.13 k-th Multiset Permutation

Given freq array $\text{cnt}[\sigma]$ and length $n = \sum \text{cnt}[i]$, find k -th lexicographic permutation (1-indexed). For each position try c in order, count per prefix via $P = \prod_i C(\text{rem} - \sum_{j < i} \text{cnt}[j], \text{cnt}[i])$, cap at $k + 1$. $O(n \cdot \sigma \cdot \log k)$.

```
string kth(vector<int> f, int n, long long k) {
    string s;
    for (int i = 0; i < n && s.size() == i; i++)
        for (int c = 0; c < (int)f.size(); c++) if (f[c]) {
            f[c]--; long long w = 1; int r = n - i - 1;
            for (int d : f) if (d) {
                for (int j = 1, m = min(d, r - d); j <= m && w
                    <= k; j++)
                    w = min<__int128>(k + 1, (__int128)w * (r - j
                    <= + 1) / j);
                r -= d;
            }
            if (w >= k) { s += 'a' + c; break; }
            k -= w; f[c]++;
        }
    return s.size() == n ? s : "";
}
```

7 Graph Theory

7.1 Floyd-Warshall

All-pairs shortest paths in $O(V^3)$. Handles negative weights. Detects and propagates negative cycles: after main DP, if $m[k][k] < 0$ then all paths through k become $-\infty$.

```
void floydWarshall(vector<vector<ll>>& m) {
    int n = m.size();
    const ll INF = 1LL << 62;
    for (int i = 0; i < n; i++)
        m[i][i] = min(m[i][i], 0LL);
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (m[i][k] != INF && m[k][j] != INF)
                    m[i][j] = min(m[i][j],
                        max(m[i][k] + m[k][j], -INF));
    for (int k = 0; k < n; k++)
        if (m[k][k] < 0)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    if (m[i][k] != INF && m[k][j] != INF)
                        m[i][j] = -INF;
}
```

7.2 Bellman-Ford / SPFA

Single-source shortest paths with negative edges. $O(VE)$. On n -th iteration, if distance still improves $\Rightarrow$ negative cycle. Sets distance to $-\infty$ for nodes reachable from negative cycle.

```
const ll LINF = LLONG_MAX / 2;
struct Edge { int a, b; ll w; };
struct BFNode { ll dist=LINF; int prev=-1; };
void bellmanFord(vector<BFNode>& nodes,
    vector<Edge>& eds, int s) {
    nodes[s].dist = 0;
```

```

int n = nodes.size();
for (int i = 0; i < n; i++) {
    bool changed = false;
    for (Edge& e : eds) {
        if (nodes[e.a].dist == LINF) continue;
        ll d = nodes[e.a].dist + e.w;
        if (d < nodes[e.b].dist) {
            nodes[e.b].dist = (i < n-1 ? d : -LINF);
            nodes[e.b].prev = e.a;
            changed = true;
        }
    }
    if (!changed) break;
}
for (int i = 0; i < n; i++)
    for (Edge& e : eds)
        if (nodes[e.a].dist == -LINF)
            nodes[e.b].dist = -LINF;
}

```

7.3 Eulerian Path (Directed)

Finds Eulerian path in directed graph. Returns empty vector if none exists. Requires in-degree == out-degree for all nodes (except start/end, diff = 1). `gr[u]` = list of (v, edge_id). `nedges` = total edges.

```

vector<int> eulerPath(
    vector<vector<pair<int,int>>>& gr,
    int nedges, int src=0) {
    int n = gr.size();
    vector<int> its(n), used(nedges), ret;
    vector<int> s = {src};
    while (!s.empty()) {
        int x = s.back(), &it = its[x];
        if (it == (int)gr[x].size()) {
            ret.push_back(x); s.pop_back();
        } else {
            auto [y,eid] = gr[x][it++];
            if (!used[eid]) { used[eid]=1; s.push_back(y); }
        }
    }
    if ((int)ret.size() != nedges+1) return {};
    reverse(ret.begin(), ret.end());
    return ret;
}

```

7.4 Graph Condense (Kosaraju SCC)

Builds condensed DAG of strongly connected components. Two DFS passes: first on original graph, second on reversed graph. `comps` = number of SCCs. `comp[u]` = component id of node `u`. Key idea: DFS1 on original records finish-order; DFS2 on reverse assigns SCCs in reverse finish-order (Kosaraju's trick).

```

struct SCC {
    int n, comps;
    vector<vector<int>> adj, rev_adj, scc_adj;
    vector<int> order, comp;
    vector<bool> vis;
}

```

```

SCC(int _n) : n(_n), comps(0), adj(_n), rev_adj(_n),
    comp(_n, -1), vis(_n, false) {}
void add_edge(int u, int v) {
    adj[u].push_back(v);
    rev_adj[v].push_back(u);
}
void dfs1(int u) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfs1(v);
    order.push_back(u);
}
void dfs2(int u, int id) {
    comp[u] = id;
    for (int v : rev_adj[u])
        if (comp[v] == -1) dfs2(v, id);
}
void build() {
    for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);
    reverse(order.begin(), order.end());
    for (int u : order) {
        if (comp[u] == -1) dfs2(u, comps++);
    }
    scc_adj.resize(comps);
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            if (comp[u] != comp[v])
                scc_adj[comp[u]].push_back(comp[v]);
        }
    }
}

```

7.5 2-SAT

Each var x_i : node $2i$ (false), $2i + 1$ (true). `either(a,b)` adds $(a \vee b)$. `setValue(x)` forces x true. `solve()` assigns `values[i]=0/1`.

```

struct TwoSat {
    int N;
    vector<vector<int>> gr;
    vector<int> values;
    TwoSat(int n=0):N(n),gr(2*n){}
    int addVar(){gr.emplace_back();gr.emplace_back();
        return N++;}
    void either(int f,int j){
        f=max(2*f,-1-2*f);j=max(2*j,-1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }
    void setValue(int x){either(x,x);}
    vector<int> val,comp,z; int time=0;
    int dfs(int i){
        int low=val[i]=++time,x;
        z.push_back(i);
        for(int e:gr[i]) if(!comp[e])
            low=min(low,val[e]?dfs(e));
        if(low==val[i]){
            do{x=z.back();z.pop_back();
                comp[x]=low;

```

```

            if(values[x>>1]==-1)values[x>>1]=x&1;
        }while(x!=i);
    }
    return val[i]=low;
}
bool solve(){
    values.assign(N,-1);
    val.assign(2*N,0);comp=val;
    for(int i=0;i<2*N;i++) if(!comp[i])dfs(i);
    for(int i=0;i<N;i++){
        if(comp[2*i]==comp[2*i+1]) return false;
        return true;
    }
};
// either(a,b)=a OR b; setValue(x)=x is true
// atMostOne({a,b,c})=at most one is true

```

7.6 Articulation Points & Bridges

Finds cut vertices and cut edges in $O(N + M)$. Root is articulation if it has ≥ 2 children in DFS tree. Edge (u,v) is a bridge if `low[v] > tin[u]`. Key idea: `low[u]` = earliest DFS entry time reachable from subtree of `u` via at most one back-edge.

```

struct Articulation {
    int n, timer, ecnt;
    vector<vector<pair<int,int>>> adj;
    vector<int> tin, low;
    vector<bool> is_art;
    vector<pair<int, int>> bridges;
    Articulation(int _n) : n(_n), timer(0), ecnt(0),
        adj(_n), tin(_n, -1), low(_n, -1),
        is_art(_n, false) {}
    void add_edge(int u, int v) {
        adj[u].push_back({v, ecnt});
        adj[v].push_back({u, ecnt++});
    }
    void dfs(int u, int peid = -1) {
        tin[u] = low[u] = timer++;
        int children = 0;
        for (auto [v, eid] : adj[u]) {
            if (eid == peid) continue;
            if (tin[v] != -1) {
                low[u] = min(low[u], tin[v]);
            } else {
                children++;
                dfs(v, eid);
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u] && peid != -1)
                    is_art[u] = true;
                if (low[v] > tin[u])
                    bridges.push_back({min(u,v), max(u,v)});
            }
        }
        if (peid == -1 && children > 1) is_art[u] = true;
    }
    void build() {
        for (int i = 0; i < n; i++)

```

```

    if (tin[i] == -1) dfs(i);
}
};

```

7.7 Biconnected Comp. & Bridge Tree

Finds 2-edge-connected components (BCCs) – maximal subgraphs with no bridges. Contract each BCC into a node; original bridges become edges of a tree (bridge tree).

How it works: First DFS finds bridges via $low[v] > tin[u]$. Second BFS floods each component avoiding bridges – all nodes reachable without crossing a bridge get the same component id.

```

struct BCC {
    int n, timer, comps;
    vector<vector<int>> adj, tree;
    vector<int> tin, low, comp;
    vector<bool> is_bridge, vis;
    vector<pair<int, int>> edges;
    BCC(int _n : n(_n), timer(0), comps(0), adj(_n),
        tin(_n, -1), low(_n, -1), comp(_n, -1),
        vis(_n, false) {}
    void add_edge(int u, int v) {
        adj[u].push_back(edges.size());
        adj[v].push_back(edges.size());
        edges.push_back({u, v});
        is_bridge.push_back(false);
    }
    void dfs(int u, int peid = -1) {
        vis[u] = true;
        tin[u] = low[u] = timer++;
        for (int id : adj[u]) {
            int v = edges[id].first ^
                edges[id].second ^ u;
            if (id == peid) continue;
            if (vis[v]) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) is_bridge[id] = true;
            }
        }
    }
    void build_tree() {
        for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);
        vector<int> q;
        for (int i = 0; i < n; i++) {
            if (comp[i] != -1) continue;
            q.push_back(i); comp[i] = comps++;
            while (!q.empty()) {
                int u = q.back(); q.pop_back();
                for (int id : adj[u]) {
                    int v = edges[id].first ^
                        edges[id].second ^ u;
                    if (comp[v] == -1 && !is_bridge[id]) {
                        comp[v] = comp[u]; q.push_back(v);
                    }
                }
            }
        }
    }
};

```

```

    }
    tree.resize(comps);
    for (int i = 0; i < (int)edges.size(); i++) {
        if (is_bridge[i]) {
            int u = comp[edges[i].first],
                v = comp[edges[i].second];
            tree[u].push_back(v);
            tree[v].push_back(u);
        }
    }
}
};

```

7.8 Euler Tour (Tree Flattening)

Key idea: $in[u]$ = DFS entry time, $out[u]$ = DFS exit time. Subtree of u = contiguous segment $[in[u], out[u]]$ in the flat array. Enables any subtree range query on a 1D array (use with BIT/Segment Tree).

```

struct EulerTour {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> in, out, flat;
    EulerTour(int _n : n(_n), timer(0), adj(_n + 1),
        in(_n + 1), out(_n + 1) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int u, int p = 0) {
        in[u] = ++timer;
        flat.push_back(u);
        for (int v : adj[u])
            if (v != p) dfs(v, u);
        out[u] = timer;
    }
    void build(int root = 1) { dfs(root); }
};

```

7.9 Binary Lifting

Preprocess $O(N \log N)$, query $O(\log N)$. $lift(u, k)$ returns k -th ancestor (or -1). $query(u, v)$ returns LCA. Call $dfs(root, -1)$ first. Key idea: $up[u][i] = 2^i$ -th ancestor of u . Lift deeper node to same depth, then binary-search highest ancestor where paths diverge.

```

struct LCA {
    int n, LOG;
    vector<vector<int>> adj, up;
    vector<int> depth;
    LCA(int _n : n(_n), LOG(__lg(_n) + 2), adj(_n),
        up(_n, vector<int>(LOG, -1)), depth(_n, 0) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int u, int p = -1, int d = 0) {
        up[u][0] = p;

```

```

        depth[u] = d;
        for (int i = 1; i < LOG; i++) {
            if (up[u][i - 1] != -1)
                up[u][i] = up[up[u][i - 1]][i - 1];
        }
        for (int v : adj[u])
            if (v != p) dfs(v, u, d + 1);
    }
    int lift(int u, int k) {
        for (int i = 0; i < LOG; i++) {
            if ((k >> i) & 1) u = up[u][i];
            if (u == -1) break;
        }
        return u;
    }
    int query(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int i = LOG - 1; i >= 0; i--) {
            if (up[u][i] != up[v][i]) {
                u = up[u][i]; v = up[v][i];
            }
        }
        return up[u][0];
    }
};

```

7.10 Heavy Light Decomposition (HLD)

Path queries on trees. Decomposes tree into $O(\log N)$ heavy paths. Call $dfs(root, 0)$ then $decompose(root, root)$. Uses BIT internally – swap with Lazy SegTree for range updates. $query(u, v)$ returns sum on path $u \rightarrow v$. $addVal(idx, delta)$ and $editVal(idx, value)$ update node value.

```

struct HLD {
    int n, cur_pos = 1;
    vector<int> arr, parent, size, depth, heavy,
        head, pos;
    vector<vector<int>> adj;
    BIT<ll> segtree;
    HLD(int n : n(n), arr(n + 1), parent(n + 1),
        size(n + 2), depth(n + 2), heavy(n + 2, 0),
        head(n + 2), pos(n + 2), adj(n + 2),
        segtree(BIT<ll>(n + 2)) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v), adj[v].push_back(u);
    }
    void dfs(int u, int p) {
        parent[u] = p; size[u] = 1;
        int max_sz = 0;
        for (int v : adj[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
            dfs(v, u);
            size[u] += size[v];
            if (size[v] > max_sz)
                max_sz = size[v], heavy[u] = v;
        }
    }
};

```

```

    }
}

void decompose(int u, int h) {
    head[u] = h; pos[u] = cur_pos++;
    segtree.add(pos[u], arr[u]);
    if (heavy[u]) decompose(heavy[u], h);
    for (int v : adj[u])
        if (v != parent[u] && v != heavy[u])
            decompose(v, v);
}

int query(int u, int v) {
    int res = 0;
    while (head[u] != head[v]) {
        if (depth[head[u]] < depth[head[v]])
            swap(u, v);
        res += segtree.sum(pos[head[u]], pos[u]);
        u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    res += segtree.sum(pos[u], pos[v]);
    return res;
}

void addVal(int idx, int delta) {
    segtree.add(pos[idx], delta);
    arr[idx] += delta;
}

void editVal(int idx, int value) {
    segtree.add(pos[idx], value - arr[idx]);
    arr[idx] = value;
}
};

```

7.11 Centroid Decomposition

Recursively finds centroids to build $O(\log N)$ depth decomposition tree. Useful for path-counting queries (e.g., count paths of length k). Mark centroid removed, recurse on remaining subtrees.

```

vector<vector<int>> adj;
vector<bool> is_removed;
vector<int> subtree_size;

int get_subtree_size(int node, int parent = -1) {
    subtree_size[node] = 1;
    for (int child : adj[node]) {
        if (child == parent || is_removed[child])
            continue;
        subtree_size[node] +=
            get_subtree_size(child, node);
    }
    return subtree_size[node];
}

int get_centroid(int node, int tree_size,
    int parent = -1) {
    for (int child : adj[node]) {
        if (child == parent || is_removed[child])
            continue;
        if (subtree_size[child] * 2 > tree_size)
            return get_centroid(child, tree_size, node);
    }
}

```

```

    return node;
}

void build_centroid_decomp(int node = 0) {
    int centroid = get_centroid(node,
        get_subtree_size(node));
    // do something with centroid
    is_removed[centroid] = true;
    for (int child : adj[centroid]) {
        if (is_removed[child]) continue;
        build_centroid_decomp(child);
    }
}

```

7.12 DSU on Tree (Sack)

$O(N \log N)$ offline subtree queries. Key idea: Process light children first (discard), then heavy child (keep). Re-add all light subtree elements at the end – each element is re-added at most $O(\log N)$ times. The `keep` flag controls whether to clear after processing.

```

int timer = 0;
vector<int> sz, in, out, flat, big;

void dfs_sz(int u, int p) {
    sz[u] = 1, big[u] = 0, in[u] = ++timer,
    flat[timer] = u;
    for (int v : adj[u]) {
        if (v != p) {
            dfs_sz(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[big[u]]) big[u] = v;
        }
    }
    out[u] = timer;
}

void add(int u, int val) { /* update DS */ }
void dfs_sack(int u, int p, bool keep) {
    for (int v : adj[u])
        if (v != p && v != big[u]) dfs_sack(v, u, 0);
    if (big[u]) dfs_sack(big[u], u, 1);
    for (int v : adj[u])
        if (v != p && v != big[u])
            for (int i = in[v]; i <= out[v]; i++)
                add(flat[i], 1);
    add(u, 1);
    // answer queries for subtree u here
    if (!keep)
        for (int i = in[u]; i <= out[u]; i++)
            add(flat[i], -1);
}

// Init: sz/in/out/flat/big.assign(n+1, 0);
// dfs_sz(1, 0); dfs_sack(1, 0, 1);

```

7.13 Hopcroft-Karp

Maximum bipartite matching. Left side size = n , right side size = m . $O(E\sqrt{V})$.

How it works: BFS builds layers of unmatched nodes, then DFS finds augmenting paths simultaneously (multi-augmenting). Each phase increases matching size by at least

1; at most $O(\sqrt{V})$ phases.

```

struct HopcroftKarp {
    int n, m;
    vector<vector<int>> g;
    vector<int> dist, pairU, pairV;
    static const int INF = 1e9;
    HopcroftKarp(int n_ = 0, int m_ = 0) {
        n = n_; m = m_;
        g.assign(n, {});
        pairU.assign(n, -1);
        pairV.assign(m, -1);
        dist.assign(n, 0);
    }

    void add_edge(int u, int v) { g[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        for (int u = 0; u < n; ++u) {
            if (pairU[u] == -1) {
                dist[u] = 0; q.push(u);
            } else dist[u] = INF;
        }
        bool foundAug = false;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                int u2 = pairV[v];
                if (u2 == -1) foundAug = true;
                else if (dist[u2] == INF) {
                    dist[u2] = dist[u] + 1; q.push(u2);
                }
            }
        }
        return foundAug;
    }

    bool dfs(int u) {
        for (int v : g[u]) {
            int u2 = pairV[v];
            if (u2 == -1 || (dist[u2] == dist[u] + 1
                && dfs(u2))) {
                pairU[u] = v; pairV[v] = u; return true;
            }
        }
        dist[u] = INF; return false;
    }

    int max_matching() {
        int matching = 0;
        while (bfs()) {
            for (int u = 0; u < n; ++u)
                if (pairU[u] == -1 && dfs(u)) ++matching;
        }
        return matching;
    }

    vector<pair<int,int>> matching_edges() const {
        vector<pair<int,int>> res;
        for (int v = 0; v < m; ++v)
            if (pairV[v] != -1)
                res.emplace_back(pairV[v], v);
    }
}

```

```
    return res;
};
```

7.14 Dinic's Algorithm (Max Flow)

$O(EV^2)$ worst case, $O(E\sqrt{V})$ for unit capacities. Edge (u, v) with capacity w is added as TWO directed arcs. $e.flow$ = current flow, $e.w$ = capacity. Access residual capacity via $e.w - e.flow$.

Key idea: BFS builds level graph; DFS sends blocking flow along it. "dead-end" nodes are skipped via `done[u]` (current arc trick).

```
const int64_t inf = 1LL << 61;
struct Dinic {
    struct edge { int to, rev;
        int64_t flow, w; };
    int n, s, t;
    vector<int> d, done;
    vector<vector<edge>> g;
    Dinic() {}
    Dinic(int _n) { n = _n + 10, g.resize(n); }
    void add_edge(int u, int v, int64_t w) {
        if (u == v) return;
        edge a = {v, (int)g[v].size(), 0, w};
        edge b = {u, (int)g[u].size(), 0, 0};
        g[u].push_back(a), g[v].push_back(b);
    }
    bool bfs() {
        d.assign(n, -1), d[s] = 0;
        queue<int> Q; Q.push(s);
        while (Q.size()) {
            int u = Q.front(); Q.pop();
            for (auto &e : g[u]) {
                int v = e.to;
                if (!d[v] && e.flow < e.w)
                    d[v] = d[u] + 1, Q.push(v);
            }
        }
        return d[t] != -1;
    }
    int64_t dfs(int u, int64_t flow) {
        if (u == t) return flow;
        for (int &i = done[u]; i < (int)g[u].size(); i++) {
            edge &e = g[u][i];
            if (e.w <= e.flow) continue;
            int v = e.to;
            if (d[v] == d[u] + 1) {
                int64_t nw = dfs(v, min(flow, e.w - e.flow));
                if (nw > 0) {
                    e.flow += nw;
                    g[v][e.rev].flow -= nw;
                    return nw;
                }
            }
        }
        return 0;
    }
};
```

```
int64_t max_flow(int _s, int _t) {
    if (_s == _t) return 0;
    s = _s, t = _t;
    int64_t flow = 0;
    while (bfs()) {
        done.assign(n, 0);
        while (int64_t nw = dfs(s, inf)) flow += nw;
    }
    return flow;
}
};
```

7.15 Min Cost Max Flow

Finds min-cost max-flow using potentials (Johnson's). $O(F \cdot E \log V)$. `addEdge(u, v, cap, cost)` adds edge $u \rightarrow v$ with capacity and cost. `maxflow(s, t)` returns minimum cost to send max flow from s to t . Supports negative costs, no negative cycles. Handles up to $\sim 10^3$ edges.

```
struct MCMF {
    struct edge { int v, rev; ll cap, cost; };
    int N;
    vector<vector<edge>> g;
    vector<ll> dist, pi;
    vector<int> preve, prevv;
    const ll INF = 1LL << 62;
    MCMF(int n) : N(n), g(n), dist(n), pi(n),
        preve(n), prevv(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
    }
    ll maxflow(int s, int t) {
        if (s == t) return 0;
        ll flow = 0, cost = 0;
        fill(pi.begin(), pi.end(), INF); pi[s] = 0;
        for (int _ = 0; _ < N; _++) {
            bool any = false;
            for (int u = 0; u < N; u++) if (pi[u] != INF)
                for (auto &e : g[u])
                    if (e.cap > 0 && pi[u] + e.cost < pi[e.v])
                        pi[e.v] = pi[u] + e.cost, any = true;
            if (!any) break;
        }
        // detect negative cycle: if any update still
        // possible, potentials are broken
        for (int u = 0; u < N; u++) if (pi[u] != INF)
            for (auto &e : g[u])
                if (e.cap > 0 && pi[u] + e.cost < pi[e.v])
                    return cost; // return 0 cost flow to avoid
                    // infinite loop
        while (true) {
            fill(dist.begin(), dist.end(), INF);
            dist[s] = 0;
            priority_queue<pair<ll, int>> pq;
            pq.push({0, s});
            while (!pq.empty()) {
                auto [d, u] = pq.top(); pq.pop();
                if (-d > dist[u]) continue;

```

```
                for (int i = 0; i < (int)g[u].size(); i++) {
                    edge &e = g[u][i];
                    ll nd = dist[u] + e.cost + pi[u] - pi[e.v];
                    if (e.cap > 0 && nd < dist[e.v]) {
                        dist[e.v] = nd; prevv[e.v] = u;
                        preve[e.v] = i;
                        pq.push({-nd, e.v});
                    }
                }
            }
            if (dist[t] == INF) break;
            for (int i = 0; i < N; i++)
                if (dist[i] < INF) pi[i] += dist[i];
            ll f = INF;
            for (int u = t; u != s; u = prevv[u])
                f = min(f, g[preve[u]][u].cap);
            for (int u = t; u != s; u = prevv[u]) {
                edge &e = g[preve[u]][u];
                e.cap -= f; g[e.v][e.rev].cap += f;
            }
            flow += f; cost += f * pi[t];
        }
        return cost;
    }
};
```

7.16 Link-Cut Tree

Dynamic tree (link/cut). Supports adding/removing edges and connectivity queries, all in $O(\log N)$ amortized. 0-indexed nodes. `link(u, v)` – add edge. `cut(u, v)` – remove edge. `connected(u, v)` – check if same tree. Each Node can store subtree data via `fix()`.

```
struct Node {
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    Node() { c[0] = c[1] = 0; fix(); }
    void fix() {
        if (c[0]) c[0] -> p = this;
        if (c[1]) c[1] -> p = this;
    }
    void pushFlip() {
        if (!flip) return;
        flip = 0; swap(c[0], c[1]);
        if (c[0]) c[0] -> flip ^= 1;
        if (c[1]) c[1] -> flip ^= 1;
    }
    int up() { return p ? p -> c[1] == this : -1; }
    void rot(int i, int b) {
        int h = i ^ b;
        Node *x = c[i], *y = b == 2 ? x : x -> c[h];
        Node *z = b ? y : x;
        if ((y -> p == p)) p -> c[up()] = y;
        c[i] = z -> c[i ^ 1];
        if (b < 2) {
            x -> c[h] = y -> c[h ^ 1];
            y -> c[h ^ 1] = x;

```

```

}
z->c[i ^ 1] = this;
fix(); x->fix(); y->fix();
if (p) p->fix();
swap(pp, y->pp);
}
void splay() {
    for (pushFlip(); p; ) {
        if (p->p) p->p->pushFlip();
        p->pushFlip(); pushFlip();
        int c1 = up(), c2 = p->up();
        if (c2 == -1) p->rot(c1, 2);
        else p->p->rot(c2, c1 != c2);
    }
}
Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
}
};
struct LinkCut {
    vector<Node> node;
    LinkCut(int n) : node(n) {}
    void link(int u, int v) {
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) {
        Node *x = &node[u], *top = &node[v];
        makeRoot(top); x->splay();
        if (x->pp) x->pp = 0;
        else { x->c[0] = top->p = 0; x->fix(); }
    }
    bool connected(int u, int v) {
        Node* nu = access(&node[u])->first();
        return nu == access(&node[v])->first();
    }
    void makeRoot(Node* u) {
        access(u); u->splay();
        if (u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0; u->fix();
        }
    }
    Node* access(Node* u) {
        u->splay();
        while (Node* pp = u->pp) {
            pp->splay(); u->pp = 0;
            if (pp->c[1])
                pp->c[1]->p = 0, pp->c[1]->pp = pp;
            pp->c[1] = u; pp->fix(); u = pp;
        }
        return u;
    }
};

```

8 Geometry

8.1 Precision Handling

Note: Never use $a == b$ for floats. Use $\text{abs}(a - b) < \text{EPS}$.

```

const double EPS = 1e-9;
bool equal(double a, double b) {
    return abs(a - b) < EPS; }
bool lessThan(double a, double b) {
    return a < b - EPS; }
bool greaterThan(double a, double b) {
    return a > b + EPS; }

```

8.2 Geometry Formulas

Triangle: Side length a, b, c Area: $\frac{1}{2} \times \text{base} \times \text{height}$ $s = \frac{a+b+c}{2}$,
 Area = $\sqrt{s(s-a)(s-b)(s-c)}$ Inradius: $r = \frac{\text{Area}}{s}$ Circum-
 radius: $R = \frac{abc}{4 \times \text{Area}}$

Circles: Arc Length: $L = \theta r$ Sector Area: $\text{Area} = \frac{1}{2} r^2 \theta$

Segment Area: $\text{Area} = \frac{1}{2} r^2 (\theta - \sin \theta)$

Rhombus: $\text{Area} = \frac{1}{2} \times d_1 \times d_2$

Trapezium: $\text{Area} = \frac{1}{2} \times (b_1 + b_2) \times h$

Regular Polygon (n sides): Perimeter = $n \times a$, Area = $\frac{1}{4} n a^2 \cot(\frac{\pi}{n})$

Circle: $C = 2\pi r$, $A = \pi r^2$

Ellipse: $A = \pi ab$

Polygon: Interior sum = $(n - 2) \times 180^\circ$

3D Shapes: **Sphere:** $SA = 4\pi r^2$, $V = \frac{4}{3} \pi r^3$

Cylinder: $SA = 2\pi r(r + h)$, $V = \pi r^2 h$

Cone: $SA = \pi r(r + l)$, $V = \frac{1}{3} \pi r^2 h$

Torus: $SA = 4\pi^2 Rr$, $V = 2\pi^2 Rr^2$

8.3 Geometry Theorems

Pick's Theorem: For a simple polygon with integer coordinates: $A = I + \frac{B}{2} - 1$, where I = interior lattice points, B = boundary lattice points.

Brahmagupta's Formula: Area of a cyclic quadrilateral with sides a, b, c, d : $S = \sqrt{(s-a)(s-b)(s-c)(s-d)}$ where $s = \frac{a+b+c+d}{2}$.

Fermat Point: Point minimizing sum of distances to triangle vertices. If no angle $> 120^\circ$, sum is $\sqrt{(a^2 + b^2 + c^2 + 4\sqrt{3} \times \text{Area})/2}$.

Simpson's Rule: $\int_a^b f(x) dx \approx \frac{h}{3} [f(x_0) + f(x_n) + 4 \sum f(x_{2i+1}) + 2 \sum f(x_{2i})]$

Pythagorean Triples: $a = 2st, b = s^2 - t^2, c = s^2 + t^2$ for $s > t > 0$, $\text{gcd}(s, t) = 1$, $s \not\equiv t \pmod{2}$.

8.4 Closest Pair of Points

Returns: Res struct containing the minimum distance squared (d) and the two points (a, b).

```

struct Res { int64_t d; P a, b; };
int64_t sq(int64_t x) { return x * x; }
int64_t dist_sq(P &a, P &b) {
    return sq(a.x - b.x) + sq(a.y - b.y);
}

```

```

Res closest_pair(vector<P>& p, vector<P>& py,
vector<P>& tmp, int l, int r) {
    int n = r - l + 1;
    if (n <= 1) return {LLONG_MAX, {}, {}};
    if (n == 2) {
        if (py[l].y > py[l + 1].y)
            swap(py[l], py[l + 1]);
        return {dist_sq(p[l], p[l + 1]), p[l], p[l + 1]};
    }
    int m = (l + r) >> 1, midx = p[m].x;
    Res res_l = closest_pair(p, py, tmp, l, m);
    Res res_r = closest_pair(p, py, tmp, m + 1, r);
    Res res = (res_l.d < res_r.d ? res_l : res_r);
    for (int i = l, j = m + 1, k = 1; k <= r; k++) {
        if (j > r || (i <= m && py[i].y < py[j].y))
            tmp[k] = py[i++];
        else tmp[k] = py[j++];
    }
    for (int i = l; i <= r; i++) py[i] = tmp[i];
    vector<P> stripe;
    for (int i = l; i <= r; i++)
        if (sq(py[i].x - midx) < res.d)
            stripe.push_back(py[i]);
    for (int i = 0; i < (int)stripe.size(); i++) {
        for (int j = i + 1; j < (int)stripe.size(); j++) {
            if (sq(stripe[i].y - stripe[j].y) >= res.d)
                break;
            int64_t d = dist_sq(stripe[i], stripe[j]);
            if (d < res.d)
                res = {d, stripe[i], stripe[j]};
        }
    }
    return res;
}

```

8.5 Rectangle Distance

Distance between the two incircle touch points of the right triangles formed by the diagonal of an $L \times W$ rectangle.

```

long double L, W;
if (!(cin >> L >> W)) return 0;
long double h = sqrtl(L*L + W*W);
long double P = L + W + h;
long double x1 = L*(L + h)/P;
long double y1 = L*W/P;
long double x2 = L*W/P;
long double y2 = W*(W + h)/P;
long double dx = x1 - x2;
long double dy = y1 - y2;
long double D = sqrtl(dx*dx + dy*dy);
cout << fixed << setprecision(6) << (double)D
<< "\n";

```

8.6 Polygon Operations

```

const ld EPS = 1e-9;
const ld PI = acos(-1.0);
// sign: -1/0/1 for negative/zero/positive
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
// 2D point with int64 coords: cross(*), dot(^),
// norm(sq), abs, rotation

```

```

struct P {
    using T = int64_t;
    T x, y;
    void read() { cin >> x >> y; }
    P operator - (const P &b) const {
        return P{x - b.x, y - b.y}; }
    P operator + (const P &b) const {
        return P{x + b.x, y + b.y}; }
    template<class U> P operator * (const U &c) const {
        return P{(T)(x * c), (T)(y * c)}; }
    template<class U> P operator / (const U &c) const {
        return P{(T)(x / c), (T)(y / c)}; }
    bool operator < (const P &b) const {
        return x == b.x ? y < b.y : x < b.x; }
    bool operator == (const P &b) const {
        return x == b.x && y == b.y; }
    T operator * (const P &b) const {
        return x * b.y - y * b.x; }
    T operator ^ (const P &b) const {
        return x * b.x + y * b.y; }
    T dot(P &a, P &b) const {
        return (a - *this) ^ (b - *this); }
    T cross(P &a, P &b) {
        return (a - *this) * (b - *this); }
    ld sq() const { return x * x + y * y; }
    ld abs() const { return sqrtl(sq()); }
    P rot() const { return P(-y, x); } // 90 CCW
    P rot(ld a) const {
        return P(x*cos(a) - y*sin(a),
            x*sin(a) + y*cos(a)); }
};
// true if a, b, c lie on same line
bool collinear(P &a, P &b, P &c) {
    return (b - a) * (c - a) == 0;
}
// true if r lies on closed segment pq
bool on_segment(P &p, P &q, P &r) {
    return collinear(p, q, r) &&
        r.x <= max(p.x, q.x) && r.x >= min(p.x, q.x) &&
        r.y <= max(p.y, q.y) && r.y >= min(p.y, q.y);
}
// Euclidean distance between a and b
ld dist_euclid(P a, P b) { return (a - b).abs(); }
// project c onto line ab (returns foot point)
P project(P a, P b, P c) {
    return a + (b - a) * (((c - a) ^ (b - a)) /
        (b - a).sq());
}
// reflect c across line ab
P reflect(P a, P b, P c) {
    return project(a, b, c) * 2 - c;
}
// perpendicular dist from c to line ab
ld dist_pt_line(P a, P b, P c) {
    return fabs1((b - a) * (c - a)) / (b - a).abs();
}
// shortest dist from c to segment ab
ld dist_pt_seg(P a, P b, P c) {

```

```

    if (((c - a) ^ (b - a)) <= 0) return (c - a).abs();
    if (((c - b) ^ (a - b)) <= 0) return (c - b).abs();
    return dist_pt_line(a, b, c);
}
// true if p lies on segment ab (EPS tolerance)
bool on_seg(P a, P b, P p) {
    return !sgn((b - a) * (p - a)) &&
        ((p - a) ^ (p - b)) <= EPS;
}
// line-line intersection (ab x cd), stores in res,
// returns false if parallel
bool line_int(P a, P b, P c, P d, P &res) {
    ld cp = (b - a) * (d - c);
    if (!sgn(cp)) return false;
    res = a + (b - a) * (((c - a) * (d - c)) / cp);
    return true;
}
// true if segment ab intersects segment cd
bool seg_int(P a, P b, P c, P d) {
    auto cp = [&](P x, P y, P z) {
        return sgn((y - x) * (z - x)); };
    if (on_seg(a, b, c) || on_seg(a, b, d) ||
        on_seg(c, d, a) || on_seg(c, d, b))
        return true;
    return cp(a, b, c) != cp(a, b, d) &&
        cp(c, d, a) != cp(c, d, b);
}
// area of polygon (shoelace), vertices in order
↔ (strict CCW or CW)
long double polygon_area(vector<P> &p) {
    int64_t area = 0, n = p.size();
    for (int i = 0; i < n; ++i)
        area += p[i] * p[(i + 1) % n];
    return (long double)abs(area) / 2.0L;
}
// convex hull vertices (CCW, monotone chain)
vector<P> convexHull(vector<P> p) {
    int n = p.size(), k = 0;
    if (n <= 2) return p;
    vector<P> h(2 * n);
    sort(p.begin(), p.end());
    for (int i = 0; i < n; h[k++] = p[i++])
        while (k >= 2 && h[k - 2].cross(h[k - 1],
            p[i]) <= 0) k--;
    for (int i = n - 2, t = k + 1; i >= 0;
        h[k++] = p[i--])
        while (k >= t && h[k - 2].cross(h[k - 1],
            p[i]) <= 0) k--;
    h.resize(k - 1);
    return h;
}
// point-in-polygon: 1=inside, 2=on boundary, 0=out
int is_point_in_polygon(const vector<P> &p, P q) {
    int c = 0;
    for (int i = 0, n = p.size(); i < n; ++i) {
        P a = p[i], b = p[(i + 1) % n];
        if ((b - a) * (q - a) == 0 &&
            min(a.x, b.x) <= q.x &&

```

```

        q.x <= max(a.x, b.x) &&
        min(a.y, b.y) <= q.y &&
        q.y <= max(a.y, b.y)) return 2;
    if ((a.y > q.y) != (b.y > q.y) &&
        ((b - a) * (q - a) > 0) == (a.y < b.y))
        c = !c;
    }
    return c;
}

```

8.7 Area Enclosed by Three Circles

Black area between three mutually tangent circles of radii r_1, r_2, r_3 (triangle area minus sector areas).

```

long double r1, r2, r3;
cin >> r1 >> r2 >> r3;
long double a = r2 + r3;
long double b = r3 + r1;
long double c = r1 + r2;
long double s = (a + b + c) / 2.0L;
long double val = s*(s - a)*(s - b)*(s - c);
long double T = val > 0 ? sqrtl(val) : 0;
auto angle = [&](long double opp, long double x,
    long double y) {
    long double cosv = (x*x + y*y - opp*opp) /
        (2.0L * x * y);
    cosv = min((long double)1.0,
        max((long double)-1.0, cosv));
    return acosl(cosv);
};

```

```

long double A = angle(a, b, c);
long double B = angle(b, c, a);
long double C = angle(c, a, b);
long double sectorSum = 0.5L*(r1*r1*A +
    r2*r2*B + r3*r3*C);
long double blackArea = T - sectorSum;
cout << fixed << setprecision(6) <<
    (double)blackArea << "\n";

```

8.8 Half-Plane Intersection

Finds the convex polygon (or empty region) satisfying all half-planes $a_i x + b_i y \leq c_i$. Lines are sorted by angle, then a deque maintains the intersection. `halfPlaneIntersection(lines)` returns vertices of the resulting polygon. $O(N \log N)$.

```

struct Line {
    double a, b, c; // ax + by <= c
    double ang;
    Line(double a = 0, double b = 0, double c = 0)
        : a(a), b(b), c(c) {
        ang = atan2(b, a);
    }
    bool operator < (const Line &o) const {
        return ang < o.ang;
    }
};
pair<double, double> intersect(Line &p, Line &q) {
    double d = p.a * q.b - p.b * q.a;

```

```

return {(p.c * q.b - q.c * p.b) / d,
        (p.a * q.c - q.a * p.c) / d};
}

bool ok(Line &a, Line &b, Line &c) {
    auto [x, y] = intersect(a, b);
    return c.a * x + c.b * y < c.c + 1e-9;
}

vector<pair<double, double>>
halfPlaneIntersection(vector<Line> v) {
    sort(v.begin(), v.end());
    vector<Line> filtered;
    for (int i = 0; i < (int)v.size(); i++) {
        if (i > 0 && abs(v[i].ang - v[i-1].ang) < 1e-9) {
            if (v[i].c < filtered.back().c) filtered.back()
                ⇨ = v[i];
        } else filtered.push_back(v[i]);
    }
    v = filtered;
    deque<Line> dq;
    for (auto &l : v) {
        while (dq.size() >= 2 &&
                !ok(dq[dq.size()-2], dq.back(), l))
            dq.pop_back();
        while (dq.size() >= 2 &&
                !ok(dq[0], dq[1], l))
            dq.pop_front();
        dq.push_back(l);
    }
    while (dq.size() >= 3 && !ok(dq[dq.size()-2],
        ⇨ dq.back(), dq[0])) dq.pop_back();
    while (dq.size() >= 3 && !ok(dq[0], dq[1],
        ⇨ dq.back())) dq.pop_front();
    vector<pair<double, double>> res;
    for (int i = 0; i < (int)dq.size(); i++)
        res.push_back(intersect(dq[i],
            dq[(i+1)%dq.size()]));
    return res;
}

```

8.9 Rotating Calipers

Extends convex hull: $O(N)$ diameter (farthest pair) and minimum width. Requires CCW hull from `convexHull`.

// Diameter: farthest pair in convex polygon, returns
⇨ actual distance

```

ld rotating_diameter(vector<P> &h) {
    int n = h.size();
    if (n <= 1) return 0;
    if (n == 2) return (h[0] - h[1]).abs();
    ld ans = 0;
    for (int i = 0, j = 1; i < n; i++) {
        while ((h[(i+1)%n] - h[i]) * (h[(j+1)%n] - h[j]) >
            ⇨ 0)
            j = (j + 1) % n;
        ans = max(ans, (h[i] - h[j]).sq());
        ans = max(ans, (h[(i+1)%n] - h[j]).sq());
    }
    return sqrtl(ans);
}

```

// Width: minimum distance between parallel supporting
⇨ lines

```

ld polygon_width(vector<P> &h) {
    int n = h.size();
    if (n <= 2) return 0;
    ld ans = 1e30;
    for (int i = 0, j = 1; i < n; i++) {
        while ((h[(i+1)%n] - h[i]) * (h[(j+1)%n] - h[j]) >
            ⇨ 0)
            j = (j + 1) % n;
        ld d = fabs1((h[(i+1)%n] - h[i]) * (h[j] - h[i]))
            / (h[(i+1)%n] - h[i]).abs();
        ans = min(ans, d);
    }
    return ans;
}

```

8.10 Point-in-Convex Polygon ($O(\log N)$)

Determines if a point is inside a CCW-ordered convex hull in $O(\log N)$.

// Returns 1 if q is inside or on boundary of convex

⇨ polygon p (CCW), 0 otherwise

```

int point_in_convex(P &q, vector<P> &p) {
    int n = p.size();
    int64_t cross_first = (p[1] - p[0]) * (q - p[0]);
    if (cross_first == 0) {
        return min(p[0].x, p[1].x) <= q.x && q.x <=
            ⇨ max(p[0].x, p[1].x) &&
            min(p[0].y, p[1].y) <= q.y && q.y <=
            ⇨ max(p[0].y, p[1].y);
    }
    if (cross_first < 0) return 0;
    int lo = 1, hi = n - 1;
    while (lo < hi) {
        int mid = (lo + hi + 1) / 2;
        if ((p[mid] - p[0]) * (q - p[0]) >= 0) lo = mid;
        else hi = mid - 1;
    }
    int64_t cross_last = (p[(lo+1)%n] - p[lo]) * (q -
        ⇨ p[lo]);
    if (cross_last < 0) return 0;
    if (cross_last == 0) {
        P a = p[lo], b = p[(lo+1)%n];
        return min(a.x, b.x) <= q.x && q.x <= max(a.x, b.x)
            ⇨ &&
            min(a.y, b.y) <= q.y && q.y <= max(a.y,
            ⇨ b.y);
    }
    return 1;
}

```

8.11 Minimum Enclosing Circle (Welzl)

Randomized expected $O(N)$. Finds smallest circle enclosing all points.

```

struct PointD {
    ld x, y;
    PointD(ld x=0, ld y=0) : x(x), y(y) {}
    PointD(P p) : x(p.x), y(p.y) {}
}

```

```

PointD operator+(PointD p) const { return {x+p.x,
    ⇨ y+p.y}; }
PointD operator-(PointD p) const { return {x-p.x,
    ⇨ y-p.y}; }
PointD operator*(ld c) const { return {x*c, y*c}; }
PointD operator/(ld c) const { return {x/c, y/c}; }
ld cross(PointD p) const { return x*p.y - y*p.x; }
ld sq() const { return x*x + y*y; }
ld abs() const { return sqrtl(sq()); }
}

```

```

struct Circle { PointD c; ld r; };
Circle welzl(vector<P> &pts) {
    mt19937 rng(chrono::steady_clock::now()
        .time_since_epoch().count());
    vector<PointD> p(pts.begin(), pts.end());
    shuffle(p.begin(), p.end(), rng);
    auto circum = [] (PointD a, PointD b, PointD c) {
        PointD ab = b-a, ac = c-a;
        ld d = 2 * ab.cross(ac);
        PointD o(
            a.x + (ab.sq()*ac.y - ac.sq()*ab.y) / d,
            a.y + (ac.sq()*ab.x - ab.sq()*ac.x) / d);
        return Circle{o, (o - a).abs()};
    };
    function<Circle(int, vector<PointD>&)> f =
        [&](int i, vector<PointD> &b) -> Circle {
        if (i == (int)p.size() || b.size() == 3) {
            if (b.empty()) return {{0,0}, 0};
            if (b.size() == 1) return {b[0], 0};
            if (b.size() == 2)
                return {(b[0]+b[1])/2, (b[0]-b[1]).abs()/2};
            return circum(b[0], b[1], b[2]);
        }
        Circle c = f(i+1, b);
        if ((c.c - p[i]).abs() > c.r + 1e-9) {
            b.push_back(p[i]);
            c = f(i+1, b);
            b.pop_back();
        }
        return c;
    };
    vector<PointD> b;
    return f(0, b);
}

```

// welzl(p).r is the radius of the minimum enclosing
⇨ circle

8.12 Minkowski Sum of Convex Polygons

$P \oplus Q = \{p + q : p \in P, q \in Q\}$ in $O(N + M)$. Result is convex. **Applications:** convex polygon collision detection, distance computation.

```

vector<P> minkowski_sum(vector<P> &a, vector<P> &b) {
    auto reorder = [] (vector<P> &p) {
        int n = p.size(), idx = 0;
        for (int i = 1; i < n; i++)
            if (p[i].y < p[idx].y ||
                (p[i].y == p[idx].y && p[i].x < p[idx].x))
                idx = i;
    };
}

```

```

    rotate(p.begin(), p.begin()+idx, p.end());
};
if (a.empty() || b.empty()) return {};
reorder(a); reorder(b);
a.push_back(a[0]); b.push_back(b[0]);
int n = a.size(), m = b.size();
vector<P> res;
for (int i = 0, j = 0; i < n || j < m; ) {
    res.push_back(a[i % n] + b[j % m]);
    if (i >= n) j++;
    else if (j >= m) i++;
    else {
        auto ca = a[(i+1)%n] - a[i%n];
        auto cb = b[(j+1)%m] - b[j%m];
        if (ca * cb >= 0) i++;
        if (ca * cb <= 0) j++;
    }
}
return res;
}

```

8.13 Circle Operations

Circumcircle, circle-line, circle-circle intersection. Uses PointD from Welzl subsection above.

// Circumcircle of three points, returns {center, radius}

```

pair<PointD, ld> circumcircle(P a, P b, P c) {
    PointD A(a), B(b), C(c);
    PointD ab = B-A, ac = C-A;
    ld d = 2 * ab.cross(ac);
    PointD o(A.x + (ab.sq()*ac.y - ac.sq()*ab.y)/d,
            A.y + (ac.sq()*ab.x - ab.sq()*ac.x)/d);
    return {o, (o - A).abs()};
}

```

// Circle-Line: returns number of intersection points

```

int circle_line(PointD c, ld r, P a, P b,
               vector<PointD> &res) {
    PointD A(a), B(b), ab = B - A;
    PointD p = A + ab * (((c-A).x * ab.x + (c-A).y *
        ↪ ab.y) / ab.sq());
    ld d = (p - c).abs();
    if (d > r + 1e-9) return 0;
    if (fabsl(d - r) < 1e-9) { res = {p}; return 1; }
    ld t = sqrtl(r*r - d*d);
    PointD dir = ab / ab.abs() * t;
    res = {p - dir, p + dir};
    return 2;
}

```

// Circle-Circle: returns number of intersection points

```

int circle_circle(PointD c1, ld r1, PointD c2, ld r2,
                 vector<PointD> &res) {
    ld d = (c1 - c2).abs();
    if (d > r1 + r2 + 1e-9 || d < fabsl(r1-r2) - 1e-9)
        return 0;
    if (d < 1e-9 && fabsl(r1-r2) < 1e-9) return -1;
    ld a = (r1*r1 - r2*r2 + d*d) / (2*d);
    ld h = sqrtl(r1*r1 - a*a);
    PointD m = c1 + (c2-c1) * (a / d);
}

```

```

PointD perp(-(c2-c1).y * h / d, (c2-c1).x * h / d);
res = {m + perp, m - perp};
return 2;
}

```

8.14 Line Sweep (Rectangle Union Area)

Union area of axis-aligned rectangles via sweep line and segment tree. $O(N \log N)$.

```

struct Event {
    ll x, y1, y2, type;
    bool operator<(const Event &o) const { return x <
        ↪ o.x; }
};
struct SegTree {
    vector<int> cnt;
    vector<ll> len, ys;
    int n;
    SegTree(vector<ll> &ys_) : ys(ys_), n(ys_.size()-1) {
        cnt.assign(4*n, 0); len.assign(4*n, 0);
    }
    void upd(int i, int l, int r, int ql, int qr, int v)
        ↪ {
        if (ql >= r || qr <= l) return;
        if (ql <= l && r <= qr) cnt[i] += v;
        else {
            int m = (l+r)/2;
            upd(i*2, l, m, ql, qr, v);
            upd(i*2+1, m, r, ql, qr, v);
        }
        if (cnt[i]) len[i] = ys[r] - ys[l];
        else len[i] = (r-l==1) ? 0 : len[i*2] + len[i*2+1];
    }
    ll total() { return len[1]; }
};
ll union_area(vector<tuple<ll,ll,ll,ll>> &rects) {
    vector<Event> ev;
    vector<ll> ys;
    for (auto [x1,y1,x2,y2] : rects) {
        ev.push_back({x1, y1, y2, 1});
        ev.push_back({x2, y1, y2, -1});
        ys.push_back(y1); ys.push_back(y2);
    }
    sort(ev.begin(), ev.end());
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end(), ys.end()), ys.end());
    SegTree seg(ys);
    ll ans = 0, prev = ev[0].x;
    for (auto &e : ev) {
        ans += (e.x - prev) * seg.total();
        int l = lower_bound(ys.begin(), ys.end(), e.y1)-
            ↪ ys.begin();
        int r = lower_bound(ys.begin(), ys.end(), e.y2)-
            ↪ ys.begin();
        seg.upd(1, 0, seg.n, l, r, e.type);
        prev = e.x;
    }
    return ans;
}

```

9 Dynamic Programming

9.1 SOS DP (Sum Over Subsets)

Complexity $O(B \cdot 2^B)$. Used for gcd/lcm/OR/AND counting. **zeta_subset**: for each mask, sum over its submasks. **zeta_superset**: for each mask, sum over its supermasks.

```

void zeta_subset(vector<int64_t> &dp, int bits) {
    for (int b = 0; b < bits; b++)
        for (int mask = 0; mask < (1LL << bits); mask++)
            if (mask >> b & 1)
                dp[mask] += dp[mask ^ (1LL << b)];
}
void zeta_superset(vector<int64_t> &dp, int bits) {
    for (int b = 0; b < bits; b++)
        for (int mask = 0; mask < (1LL << bits); mask++)
            if (!(mask >> b & 1))
                dp[mask] += dp[mask | (1LL << b)];
}
void Solve() {
    ll n; cin >> n;
    vector<ll> arr(n);
    for (auto& x : arr) cin >> x;
    ll N = 1 + __lg(*max_element(arr.begin(),
        arr.end()));
    vector<ll> sos(1LL << N, 0), sos2(1LL << N, 0);
    for (auto it : arr) {
        sos[it]++;
        sos2[((1LL << N) - 1) ^ it]++;
    }
    for (int i = 0; i < N; ++i)
        for (int mask = 0; mask < (1LL << N); ++mask)
            if (mask & (1LL << i))
                sos[mask] += sos[mask ^ (1LL << i)],
                sos2[mask] += sos2[mask ^ (1LL << i)];
    for (auto& it : arr) {
        // count y: x/y == x
        cout << sos[it] << ' ';
        // count y: x&y == x
        cout << sos2[((1LL << N) - 1) ^ it] << ' ';
        // count y: x&y != 0
        cout << n - sos2[((1LL << N) - 1) ^ it] << ' ';
    }
}

```

9.2 Knuth Optimization

For DP on intervals where optimal split point $p[i][j]$ is monotone in both i and j : $dp[i][j] = \min_{i < k < j} (dp[i][k] + dp[k][j]) + f(i, j)$ Reduces $O(N^3)$ to $O(N^2)$. **Note:** Speeds up interval DP from $O(N^3)$ to $O(N^2)$ if costs are monotonic.

```

for (int i = 0; i <= n; i++) {
    p[i][i] = i, dp[i][i] = 0;
    if (i < n) dp[i][i+1] = cost(i, i+1), p[i][i+1] = i;
}
for (int len = 2; len <= n; len++) {
    for (int i = 0; i + len <= n; i++) {
        int j = i + len;
        dp[i][j] = LLONG_MAX;
    }
}

```

```

for (int k = p[i][j-1]; k <= p[i+1][j]; k++) {
    ll val = dp[i][k] + dp[k][j] + cost(i, j);
    if (val < dp[i][j]) {
        dp[i][j] = val;
        p[i][j] = k;
    }
}
}
}

```

9.3 Max Rectangular Sum in Matrix

Key idea: Fix top and bottom rows, collapse 2D into 1D column sums, then apply Kadane's algorithm for max subarray. $O(N^2M)$.

```

int64_t maxRectSum(vector<vector<int>>& a) {
    int n = a.size(), m = a[0].size();
    int64_t ans = -4e18; // Safe negative infinity
    for (int top = 0; top < n; top++) {
        vector<int64_t> col(m, 0);
        for (int bot = top; bot < n; bot++) {
            for (int j = 0; j < m; j++)
                col[j] += a[bot][j];
            int64_t cur = 0;
            for (int j = 0; j < m; j++) {
                cur = max(col[j], cur + col[j]);
                ans = max(ans, cur);
            }
        }
    }
    return ans;
}

```

9.4 Subset Sum (Bitset)

Works when sum of values $\leq N$. $O(N\sqrt{N}/64)$. Uses bitset DP with binary splitting optimization. `dp.test(k)` is true if sum k can be formed.

```

const int N = 1e6 + 6;
bitset<N> dp;
dp.set(0);
for (auto &[w, c] : comp) {
    for (int i = 0; (1LL << i) <= c; ++i) {
        dp |= dp << (w * (1LL << i));
        c -= 1LL << i;
    }
    dp |= dp << (w * c);
}
dp.test(k) // if sum k is possible

```

OR convolution can be done using SOS DP.

9.5 Exclusion DP (GCD/LCM counting)

Count pairs/ k -tuples with given gcd. Let $d[i]$ = count of elements divisible by i . Then:

Ordered pairs (a, b) , $(a, b) \neq (b, a)$: $f[i] = d[i] \cdot d[i]$

Unordered (with rep): $f[i] = d[i](d[i] + 1)/2$

Unordered (no rep): $f[i] = d[i](d[i] - 1)/2$

Ordered k -tuples: $f[i] = (d[i])^k$

Unordered (no rep): $f[i] = \binom{d[i]}{k}$

Unordered (with rep): $f[i] = \binom{d[i]+k-1}{k}$

9.6 Li Chao Tree (CHT)

Maintains a set of lines $y = mx + b$, answers minimum y -value at any query point x . $O(\log C)$ per insert/query, C = coordinate range.

When to use: DP transitions of the form $dp[j] = \min_i(m_i \cdot x_j + b_i)$ where slopes/intercepts come from earlier states. Nodes allocated lazily. For maximum queries, init with `LLONG_MIN` and flip comparisons. **Note:** For queries like $y = mx + b$. Does not require queries or lines to be sorted.

```

struct LiChao {
    struct Line {
        ll m, b;
        ll get(ll x) { return m * x + b; }
    };
    struct Node { int l, r; Line line; };
    vector<Node> seg;
    LiChao() { seg.push_back({-1, -1, {0, LLONG_MAX}}); }
    void addLine(Line nw, int idx = 0,
                  ll l = -1e9, ll r = 1e9) {
        ll m = l + (r - l) / 2;
        bool left = nw.get(l) < seg[idx].line.get(l);
        bool mid = nw.get(m) < seg[idx].line.get(m);
        if (mid) swap(seg[idx].line, nw);
        if (l == r) return;
        if (left != mid) {
            if (seg[idx].l == -1) {
                seg[idx].l = seg.size();
                seg.push_back({-1, -1, {0, LLONG_MAX}});
            }
            addLine(nw, seg[idx].l, l, m);
        } else {
            if (seg[idx].r == -1) {
                seg[idx].r = seg.size();
                seg.push_back({-1, -1, {0, LLONG_MAX}});
            }
            addLine(nw, seg[idx].r, m + 1, r);
        }
    }
    ll query(ll x, int idx = 0, ll l = -1e9, ll r = 1e9)
    ↪ {
        if (idx == -1) return LLONG_MAX;
        ll m = l + (r - l) / 2;
        ll res = seg[idx].line.get(x);
        if (l != r) {
            if (x <= m) res = min(res, query(x, seg[idx].l,
            ↪ l, m));
            else res = min(res, query(x, seg[idx].r, m+1,
            ↪ r));
        }
        return res;
    }
}
// LiChao lct; lct.addLine({slope, intercept});
// ll ans = lct.query(x);

```

10 Others

10.1 Bit Hacks

```

// --- Basic ops ---
x & -x // LSB value
x & (x - 1) // unset LSB
x | (x - 1) // set trailing 0s to 1
~x & (x + 1) // isolate lowest 0-bit
x && !(x & (x - 1)) // is power of two?
x >> i & 1 // get bit i
x |= (1 << i) // set bit i
x &= ~(1 << i) // clear bit i
x ^= (1 << i) // toggle bit i
(1 << k) - 1 // mask of k lowest
↪ bits
// --- GCC builtins (suffix ll for 64-bit; undef for 0)
↪ ---
__builtin_popcount(x) // # of 1-bits
__builtin_clz(x) // leading zeros
__builtin_ctz(x) // trailing zeros
__builtin_parity(x) // popcount mod 2
// --- MSB / bit width (32-bit) ---
31 - __builtin_clz(x) // MSB position (0-idx)
32 - __builtin_clz(x) // bit width
1 << (31 - __builtin_clz(x)) // MSB value
// --- Iterate set bits of x ---
for (int s = x; s; s &= s - 1) {
    int i = __builtin_ctz(s); // current bit index
}
// --- Iterate submasks of mask m ---
for (int s = m; ; s = (s - 1) & m) {
    if (s == 0) break;
}
// --- Next number with same popcount ---
int next_popcount(int x) {
    int c = x & -x, r = x + c;
    return r | (((r ^ x) >> 2) / c);
}
// --- Round up to power of two ---
uint32_t next_pow2(uint32_t n) {
    n--; n |= n >> 1; n |= n >> 2; n |= n >> 4;
    n |= n >> 8; n |= n >> 16; return ++n;
}
// 64-bit: add n |= n >> 32
// --- Gray code ---
int gray = x ^ (x >> 1); // binary -> Gray
// Gray -> binary: g = gray; for (int m = g >> 1; m;
↪ m >>= 1) g ^= m;

```

10.2 Zobrist Hashing (XOR Hashing)

Each element gets a random 64-bit value via `mt19937_64`. Hash of a set = XOR of its elements' hashes. $O(1)$ add/remove, collision probability negligible ($\approx 2^{-64}$).

Applications:

- Subarray anagram detection:** Assign random $rnd[v]$ to each distinct value. Prefix XOR $pre[i + 1] = pre[i] \oplus rnd[a[i]]$. Two subarrays $[l, r]$ and $[l', r']$ are

anagrams iff $pre[r+1] \oplus pre[l] = pre[r'+1] \oplus pre[l']$.

- **Set equality:** Hash two sets by XORing element hashes; equal $\implies$ equal hash.
- **Tree isomorphism:** $hash[u] = rnd[val[u]] \oplus \bigoplus_{v \in children} hash[v]$ (sort child hashes first for unordered trees).
- **Game state hashing:** Chess/Go engines assign random numbers to each piece+square; board hash = XOR of all active pieces (Zobrist origin).

```
mt19937_64 rng(chrono::steady_clock::now()
    .time_since_epoch().count());
vector<uint64_t> rnd(N);
for (auto &x : rnd) x = rng();
```

```
// Prefix XOR for subarray hash
vector<uint64_t> pre(n + 1, 0);
for (int i = 0; i < n; i++)
    pre[i + 1] = pre[i] ^ rnd[a[i]];
// xor of range [l, r] = pre[r+1] ^ pre[l]
```

10.3 Range OR, XOR, AND Tricks

```
int64_t xor_0ton(int64_t n) {
    if (n % 4 == 0) return n;
    if (n % 4 == 1) return 1;
    if (n % 4 == 2) return n + 1;
    return 0;
}

int64_t range_xor(int64_t l, int64_t r) {
    return xor_0ton(r) ^ xor_0ton(l - 1);
}

int64_t range_and(int64_t l, int64_t r) {
    int b = 0;
    while ((l >> b) ^ (r >> b)) b += 1;
    return (l >> b) << b;
}

int64_t range_or(int64_t l, int64_t r) {
    int b = 0;
    while ((l >> b) ^ (r >> b)) b += 1;
    return r | ((1LL << b) - 1);
}
```

10.4 Inclusion–Exclusion with Bitmasks

Count elements satisfying **none** of n properties $P_1, \dots, P_n$:

$$\left| \bigcap_{i=1}^n \overline{P_i} \right| = \sum_{mask=0}^{2^n-1} (-1)^{\text{popcount}(mask)} \cdot f(mask)$$

where $f(mask)$ = count of elements satisfying all properties with bit = 1 in mask.

```
template<typename F>
ll inclusion_exclusion(int n, F f) {
    ll ans = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
```

```
        ll sign = (__builtin_popcount(mask) & 1) ? -1 : 1;
        ans += sign * f(mask);
    }
    return ans;
}
// Count numbers in [1,N] not divisible by any prime in
// p[0..n-1]
// auto f = [&](int mask) -> ll {
//     ll prod = 1;
//     for (int i = 0; i < n; i++)
//         if (mask >> i & 1) prod *= p[i];
//     return N / prod;
// };
// ll result = inclusion_exclusion(n, f);
```

10.5 Sum of Digits up to N

Counts total digit sum of all integers $[1, N]$. Key idea: For each decimal position with factor f , count contribution of each digit value (0-9) to that position across all numbers. $O(\log N)$.

```
uint64_t sumDigitsUpTo(uint64_t n) {
    if (n <= 0) return 0;
    uint64_t sum = 0;
    for (uint64_t f = 1; f <= n; f *= 10) {
        uint64_t hi = n / (f * 10);
        uint64_t cur = (n / f) % 10;
        uint64_t lo = n % f;
        sum += hi * 45LL * f;
        sum += cur * (lo + 1);
        sum += f * (cur * (cur - 1) / 2);
    }
    return sum;
}
```

10.6 Number of Digits up to N

Total count of digits written out in $[1, N]$ (e.g. for $N = 12$: $1, 2, \dots, 9, 1, 0, 1, 1, 1, 2 = 15$ digits). Key: Numbers with d digits span a range; count their contribution per digit-length. $O(\log N)$.

```
uint64_t get(uint64_t n) {
    if (n <= 0) return 0;
    uint64_t base = 1, digits = 1, now = 0;
    while (base <= n / 10) {
        base *= 10;
        now += (base - base / 10) * digits;
        digits += 1;
    }
    n -= (base - 1);
    now += n * digits;
    return now;
}
```

10.7 Game Theory & Nim

Standard Nim: XOR sum of piles = 0 $\implies$ Second player wins. $\neq 0 \implies$ First player wins.

Subtraction Game: Can only take up to k stones. XOR sum of $(pile_i \bmod (k+1))$.

Index-k Nim: Can remove from up to k piles at once.

Write sizes in binary. Sum each column. If all column sums $\equiv 0 \pmod{k+1}$, second player wins.

Misère Nim (Last player to move LOSES): If all piles have size 1, Second wins if N is ODD. Otherwise, play like Standard Nim (XOR sum).

10.8 Sprague-Grundy

$g(v) = \text{mex}\{g(u) : u \text{ reachable from } v\}$. $g = 0 \implies$ losing, else winning. XOR combines sub-games, mex computes single state.

```
// DP template (bool[] avoids set overhead)
int g[N]; g[0] = 0;
for (int i = 1; i <= maxN; i++) {
    bool seen[K] = {};
    for (int m : moves) if (i >= m) seen[g[i - m]] = 1;
    int m = 0; while (seen[m]) m++;
    g[i] = m;
}
// Standard Nim: g(n) = n (XOR pile sizes / 1st wins if != 0)
// Subtraction {1..m}: g(n) = n % (m+1)
// Take & Break: g(n) = mex{ g(a) ^ g(b) | a+b=n }
// Staircase Nim: XOR odd-indexed piles only
// Wythoff: losing pairs = (floor(phi*k), floor(phi^2*k))
// Pattern: DP 100 terms for periodicity.
// g >> 1000 => closed form.
// Watch: mex includes 0; g(0)=0; add moves irrelevant;
// count losing via pairs XOR=0
```